

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

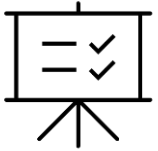
GRAPH TRAVERSALS ALGORITHMS: BFS AND DFS

AIM OF THE SESSION



To familiarize students with the concept of graph traversal techniques.

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- graph traversal techniques.
2. Describe :- DFS and BFS.

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- graph traversal techniques.
2. Describe :- DFS and BFS.
3. Summarize:- graph traversal techniques.

Graph Traversal

- Graph traversal is a technique used for searching a vertex in a graph.
- The graph traversal is also used to decide the order of vertices is visited in the search process.
- A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of the graph without getting into looping path.
- There are two graph traversal techniques and they are as follows...
 1. DFS (Depth First Search)
 2. BFS (Breadth First Search)

Breadth First Search

- **BFS** Stands for Breadth First Search. It is also known as level order traversal.
- The Queue data structure is used for the Breadth First Search traversal.
- When we use the BFS algorithm for the traversal in a graph, we can consider any node as a root node.
- BFS traversal of a graph produces a spanning tree as final result. Spanning Tree is a graph without loops.

Breadth First Search Algorithm

Step 1 - Define a Queue of size total number of vertices in the graph.

Step 2 - Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.

Step 3 - Visit all the non-visited adjacent vertices of the vertex which is at front of the Queue and insert them into the Queue.

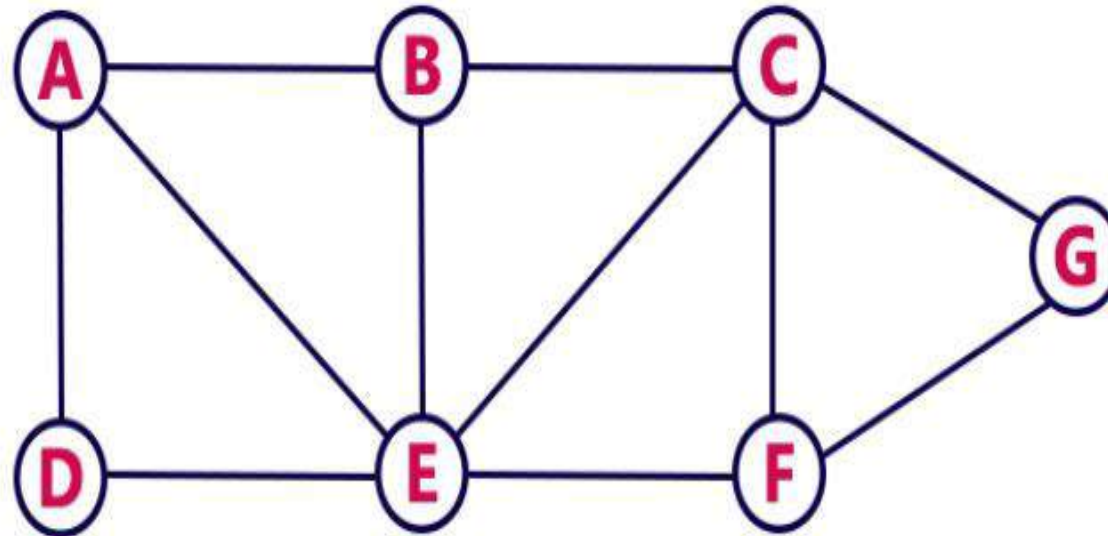
Step 4 - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.

Step 5 - Repeat steps 3 and 4 until queue becomes empty.

Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

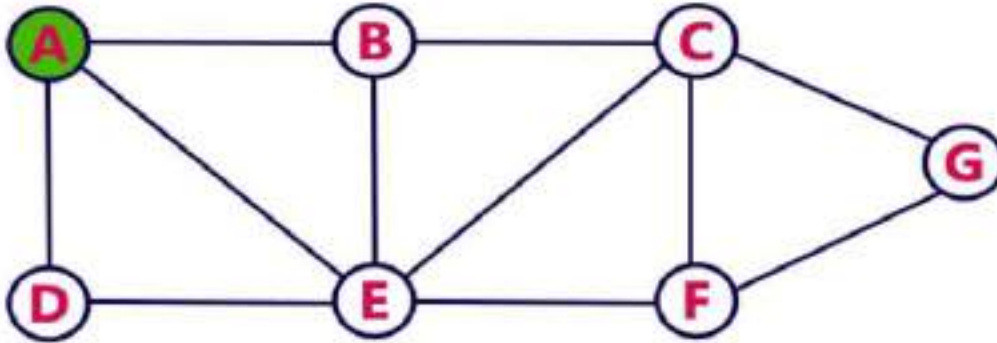
Example

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

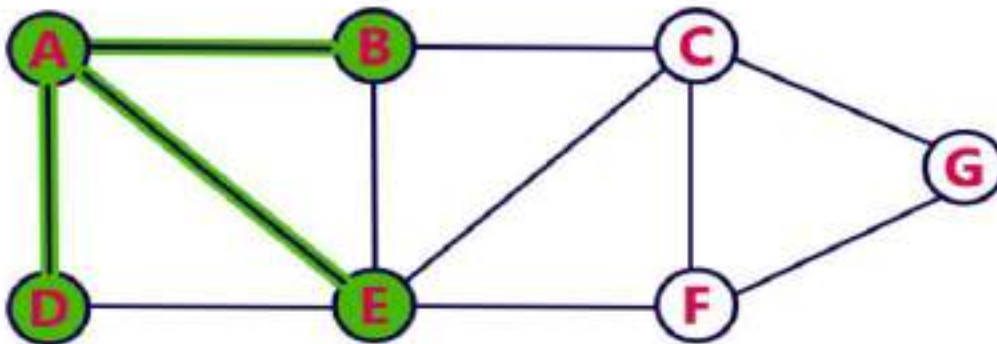


Queue



Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

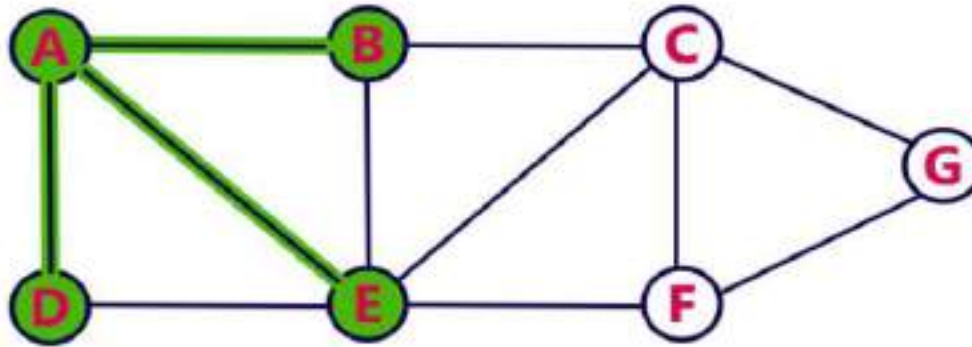


Queue



Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

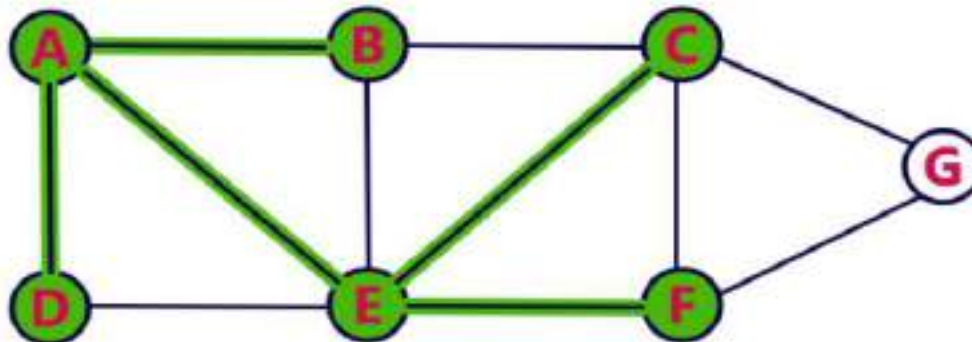


Queue



Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

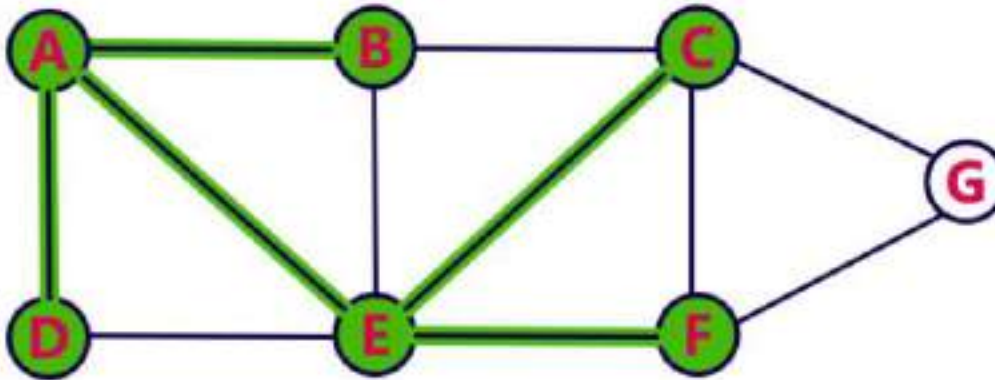


Queue



Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

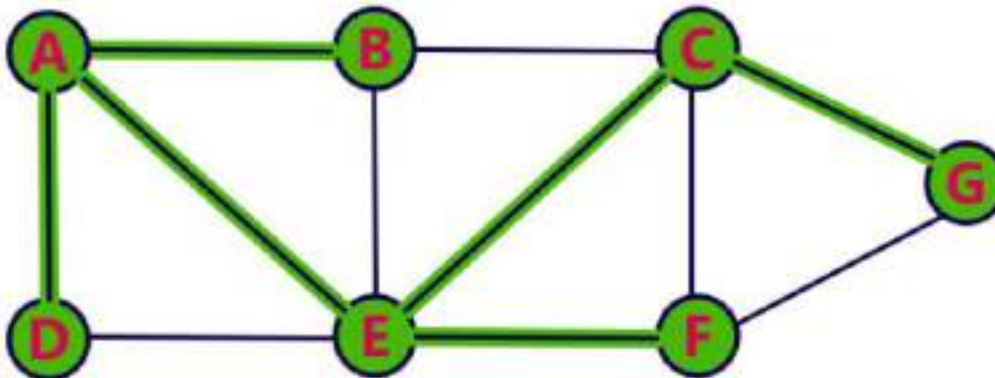


Queue



Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.

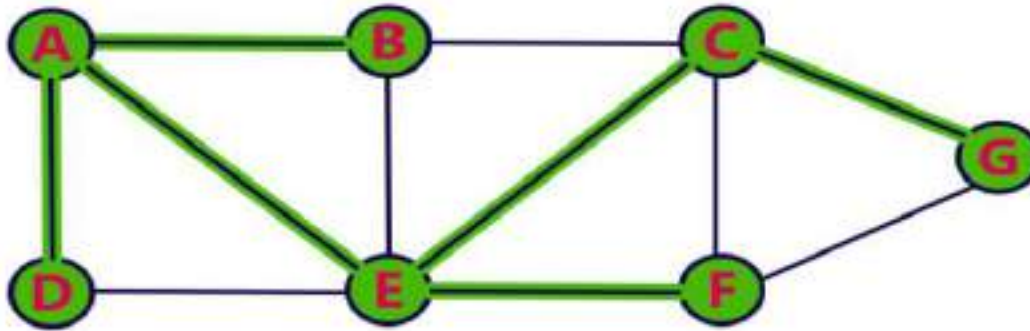


Queue



Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

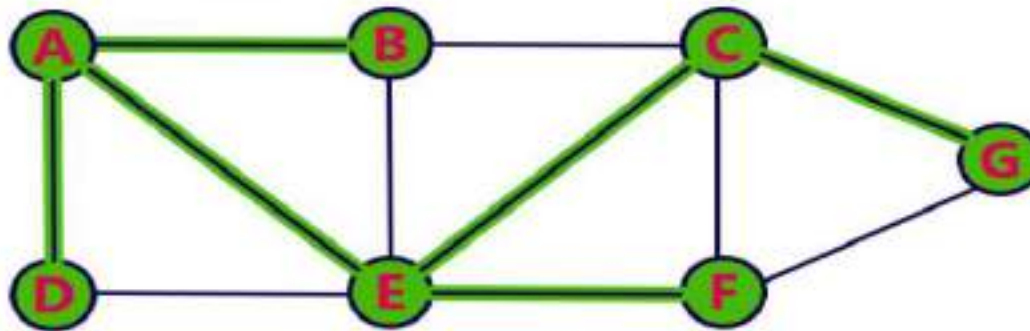


Queue



Step 8:

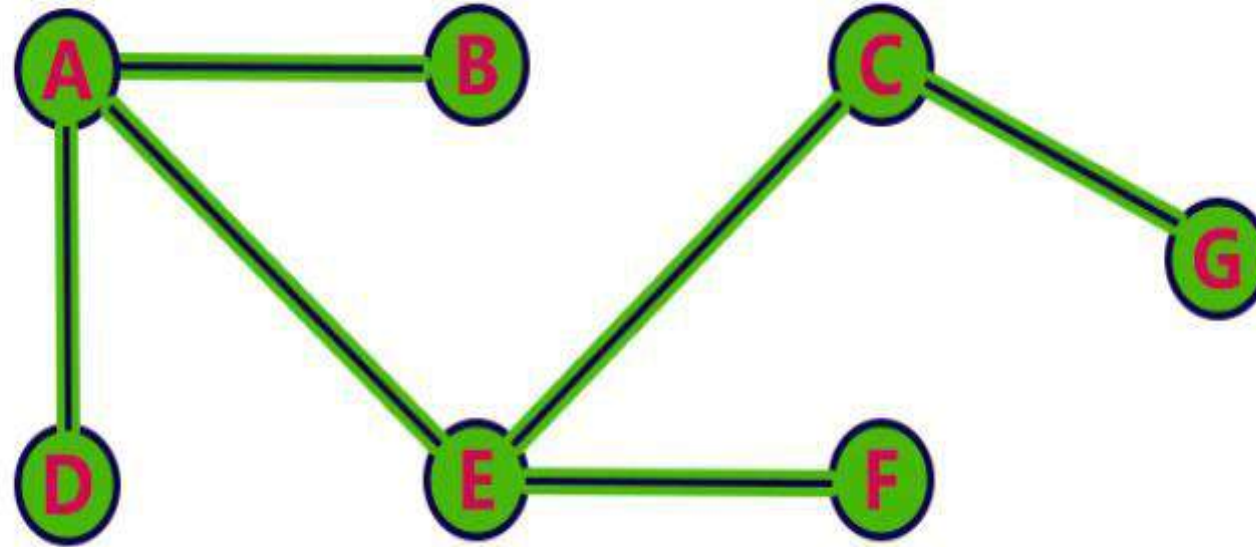
- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue



- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



Depth First Search

- **DFS traversal of a graph produces a spanning tree as final result.**
- **Spanning Tree is a graph without loops.**
- **We use Stack data structure with maximum size of total number of vertices in the graph to implement DFS traversal.**
- **It is also called as Preorder Traversal.**

Depth First Search Algorithm

Step 1 - Define a Stack of size total number of vertices in the graph.

Step 2 - Select any vertex as starting point for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack.

Step 4 - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

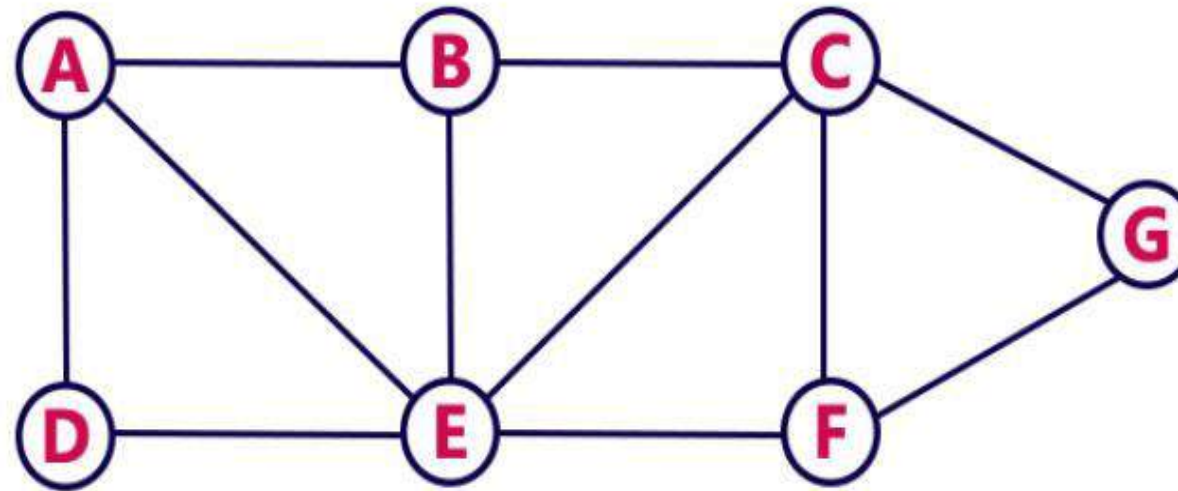
Step 5 - When there is no new vertex to visit then use back tracking and pop one vertex from the stack.

Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

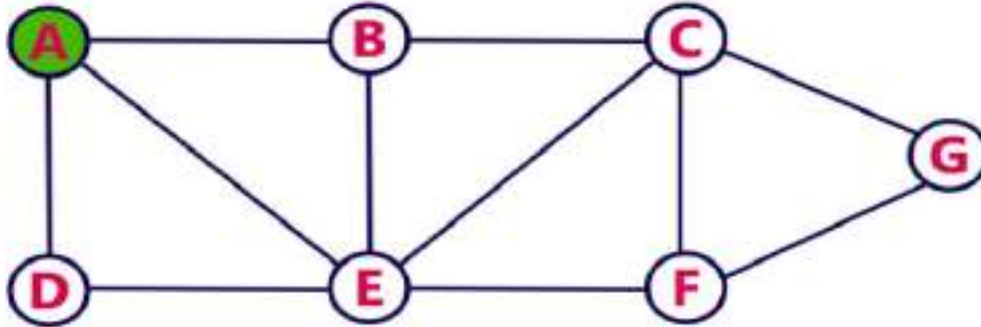
Example

Consider the following example graph to perform DFS traversal



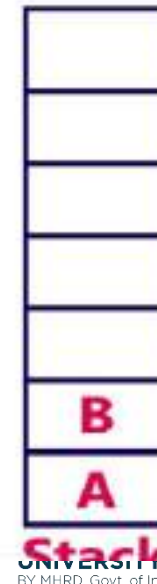
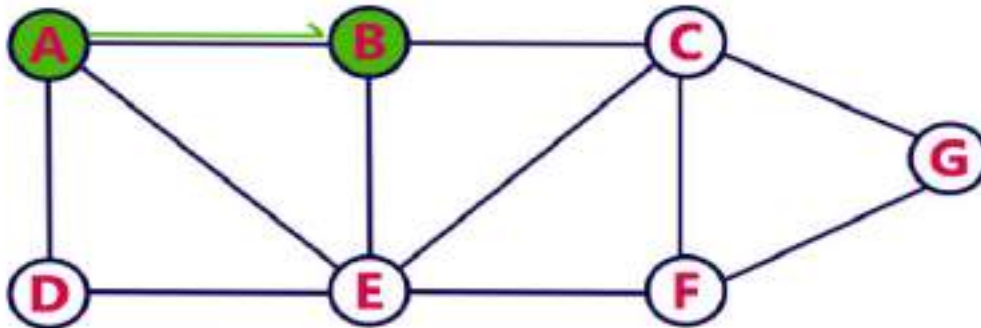
Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



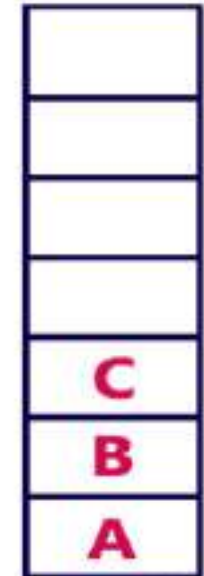
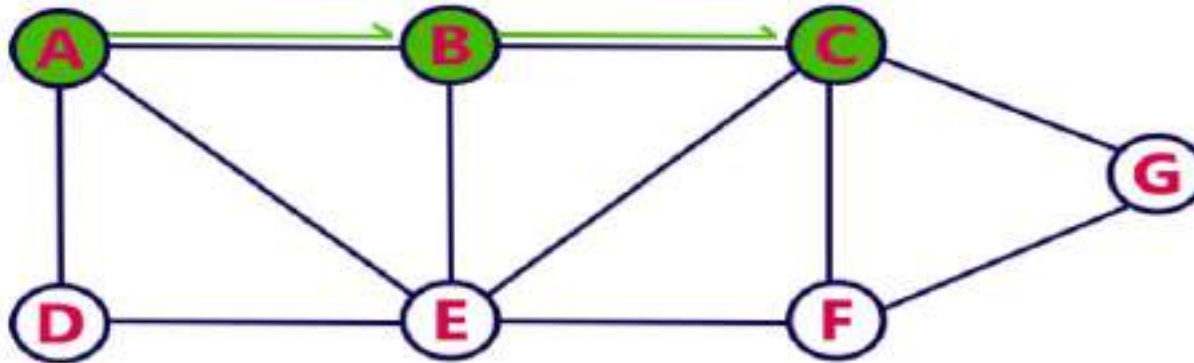
Step 2:

- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.



Step 3:

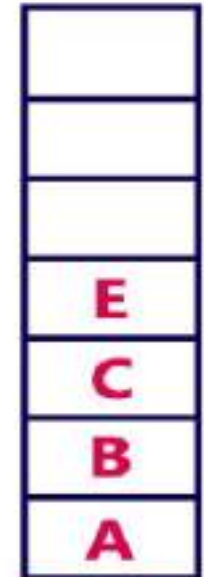
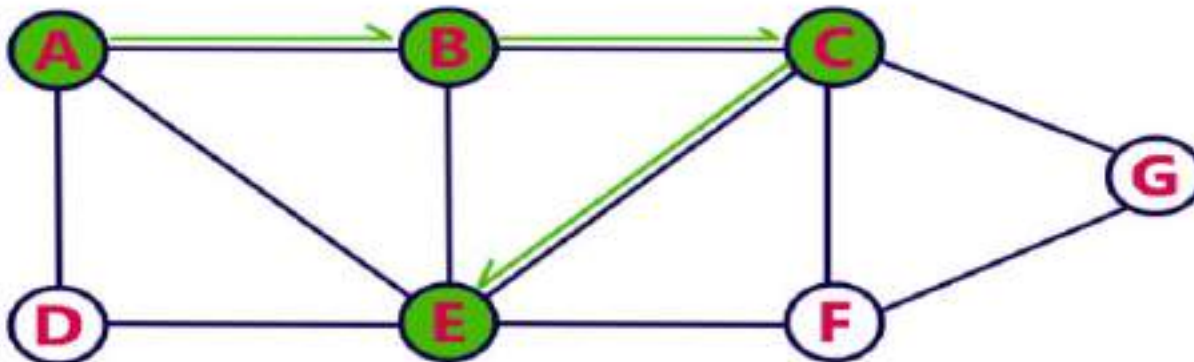
- Visit any adjacent vertex of **B** which is not visited (**C**).
- Push C on to the Stack.



Stack

Step 4:

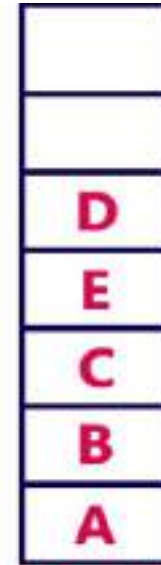
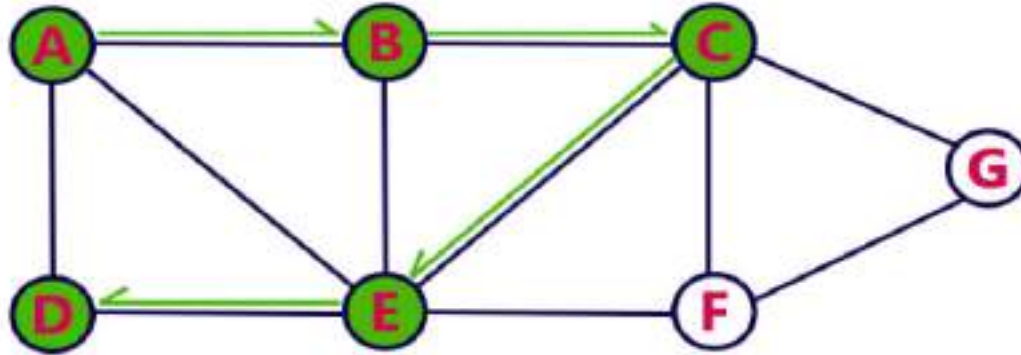
- Visit any adjacent vertex of **C** which is not visited (**E**).
- Push E on to the Stack



Stack

Step 5:

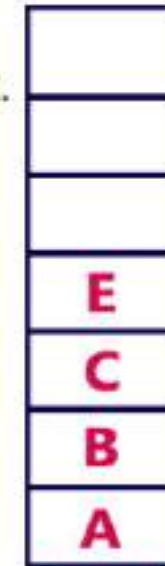
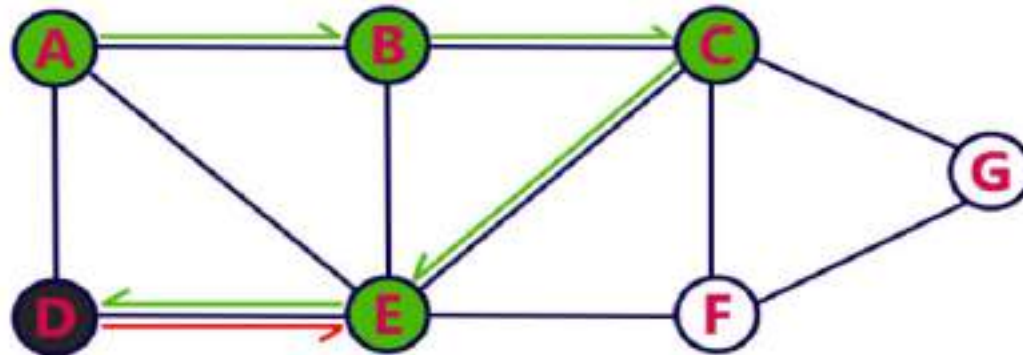
- Visit any adjacent vertex of **E** which is not visited (**D**).
- Push D on to the Stack



Stack

Step 6:

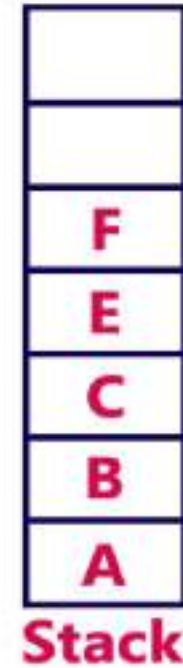
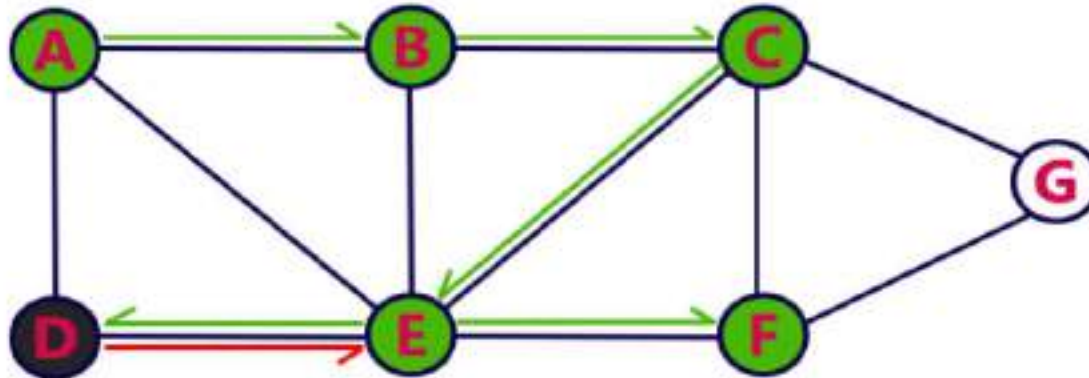
- There is no new vertex to be visited from D. So use back track.
- Pop D from the Stack.



Stack

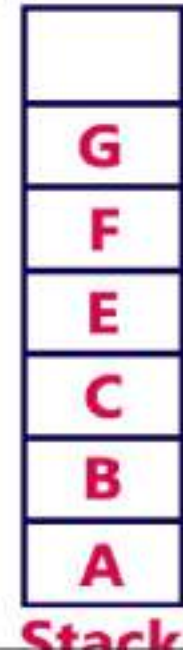
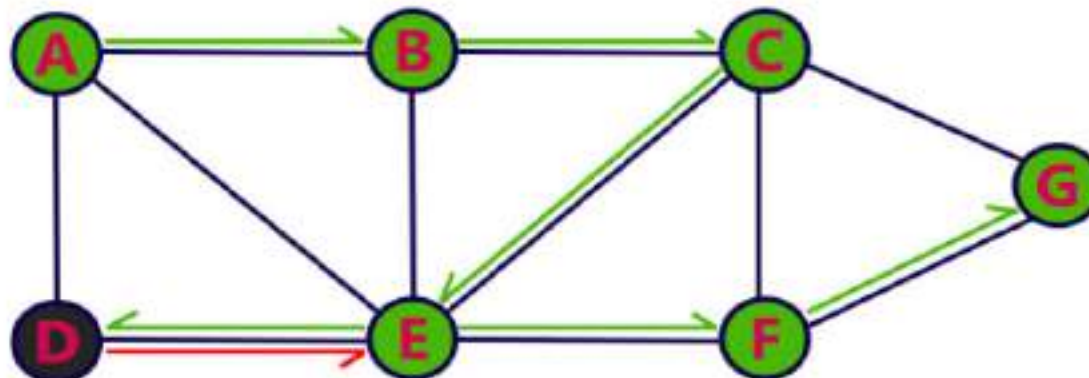
Step 7:

- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.



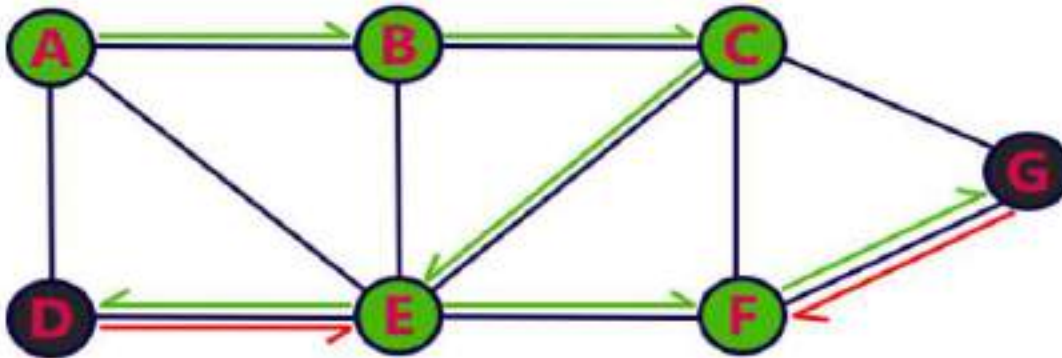
Step 8:

- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.

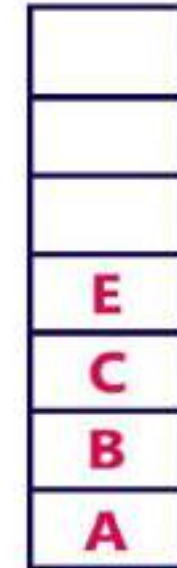


Step 9:

- There is no new vertex to be visited from G. So use back track.
- Pop G from the Stack.

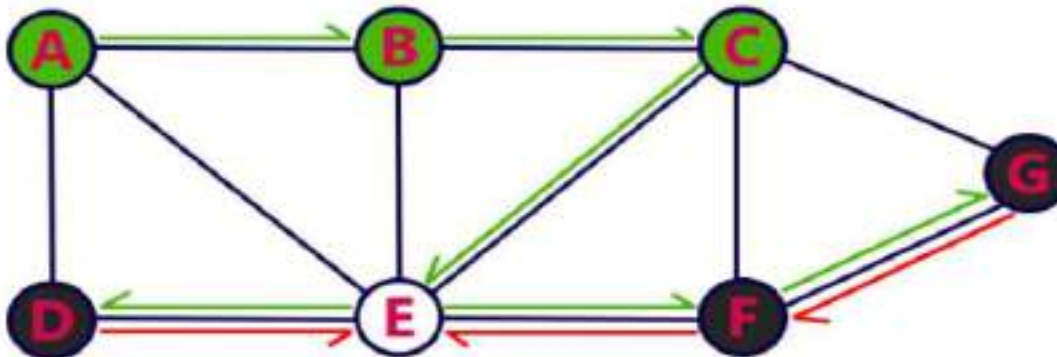


Stack

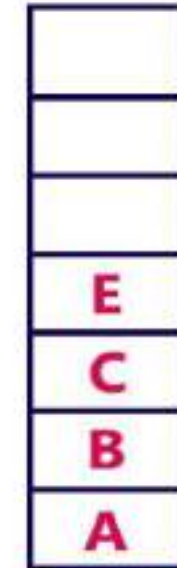


Step 10:

- There is no new vertex to be visited from F. So use back track.
- Pop F from the Stack.

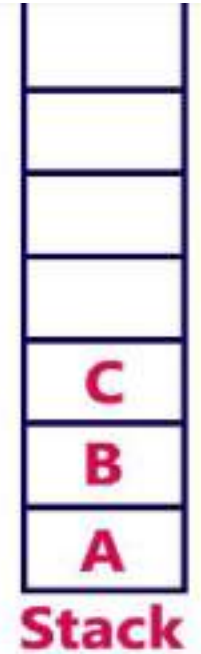
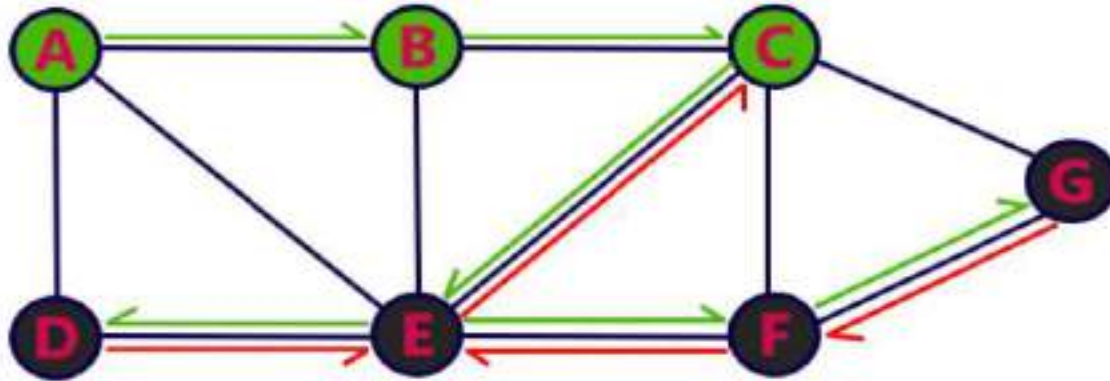


Stack



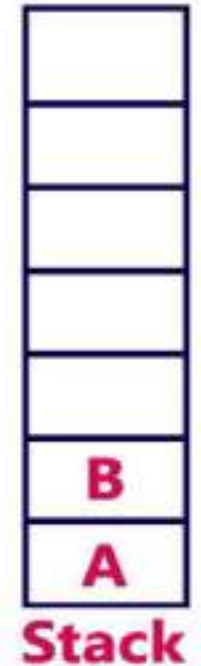
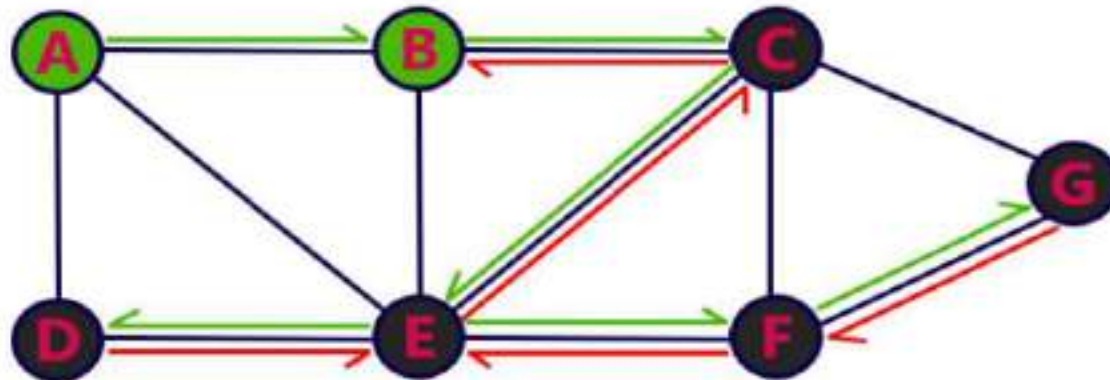
Step 11:

- There is no new vertex to be visited from E. So use back track.
- Pop E from the Stack.



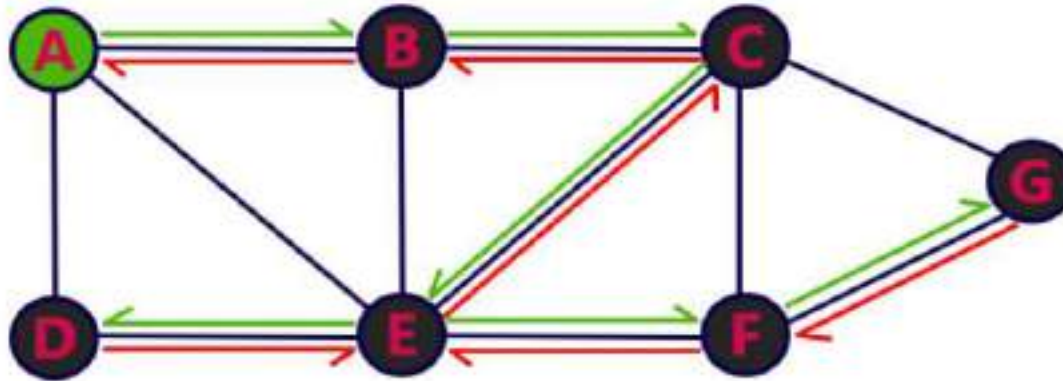
Step 12:

- There is no new vertex to be visited from C. So use back track.
- Pop C from the Stack.



Step 13:

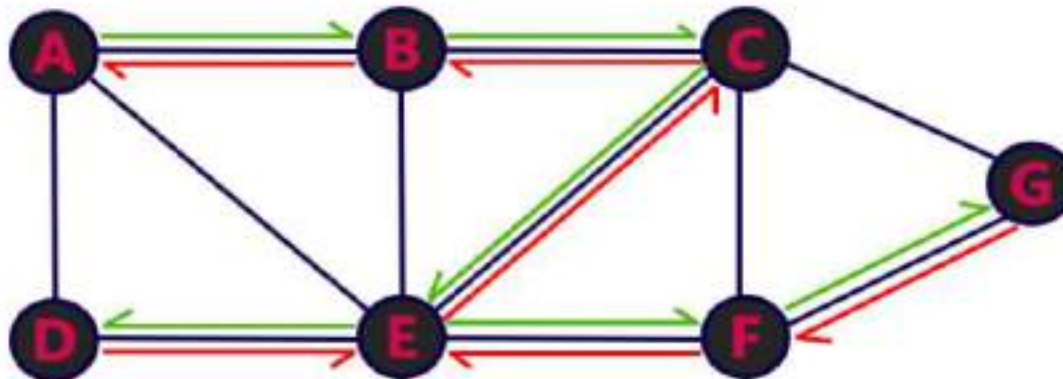
- There is no new vertex to be visited from B. So use back track.
- Pop B from the Stack.



A
Stack

Step 14:

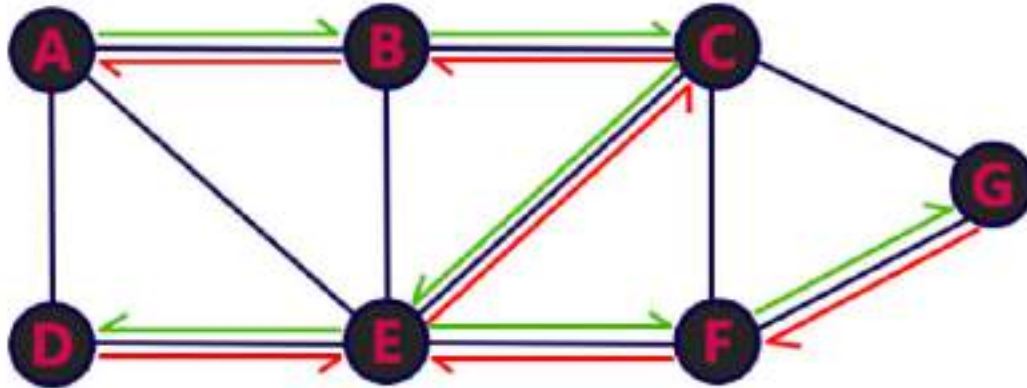
- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.



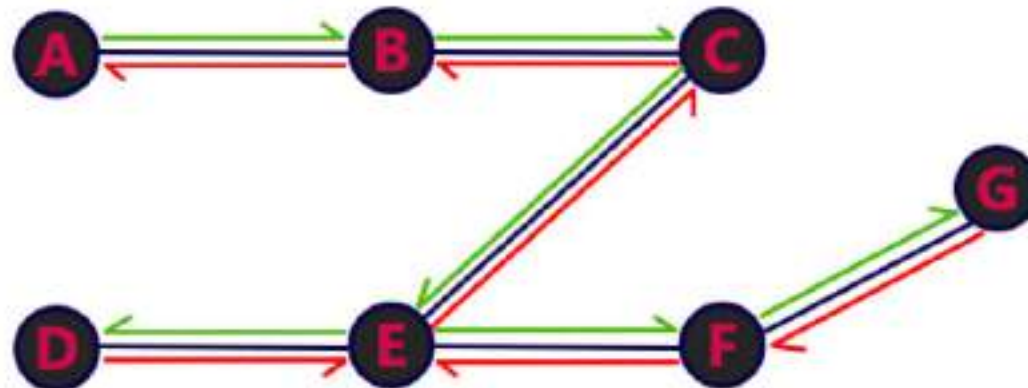
Stack

Step 14:

- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.



- Stack became Empty. So stop DFS Traversal.
- Final result of DFS traversal is following spanning tree.



Time Complexity of BFS and DFS

- Time Complexity of BFS = $O(V+E)$ where V is vertices and E is edges.
- Time Complexity of DFS is also $O(V+E)$ where V is vertices and E is edges.

A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of the graph without getting into looping path.

SELF-ASSESSMENT QUESTIONS

- What is the time complexity of the Ford-Fulkerson algorithm in the worst case if implemented with the breadth-first search (BFS)?

A. ($O(V + E)$)

B. ($O(V^2E)$)

C. ($O(VE^2)$)

D. ($O(V^3)$)

- What does the Ford-Fulkerson algorithm use to find the augmenting path?

A. Depth-First Search (DFS)

B. Breadth-First Search (BFS)

C. Either DFS or BFS

D. Dijkstra's Algorithm

TERMINAL QUESTIONS

- **What are BFS and DFS with examples?**
- **How to choose between BFS and DFS?**
- **Which is faster, BFS or DFS?**

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

- 1 Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professional.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Dijkstra Algorithm

- It is used for solving the single source shortest path problem.
- It computes the shortest path from one particular source node to all other remaining nodes of the graph.
- Dijkstra algorithm works only for connected graphs.
- It only provides the value or cost of the shortest paths.
- Dijkstra algorithm works for directed as well as undirected graphs.

Dijkstra's Algorithm

- Dijkstra's algorithm is an iterative algorithm that finds the **shortest path** from **source vertex** to all **other vertices** in the graph.
- It does not work on graphs with **negative weights**(or edges).
- It is a **greedy algorithm** because it always relay on local optimum.
- **Applications:** Maps, computer networking(Routing systems) etc..

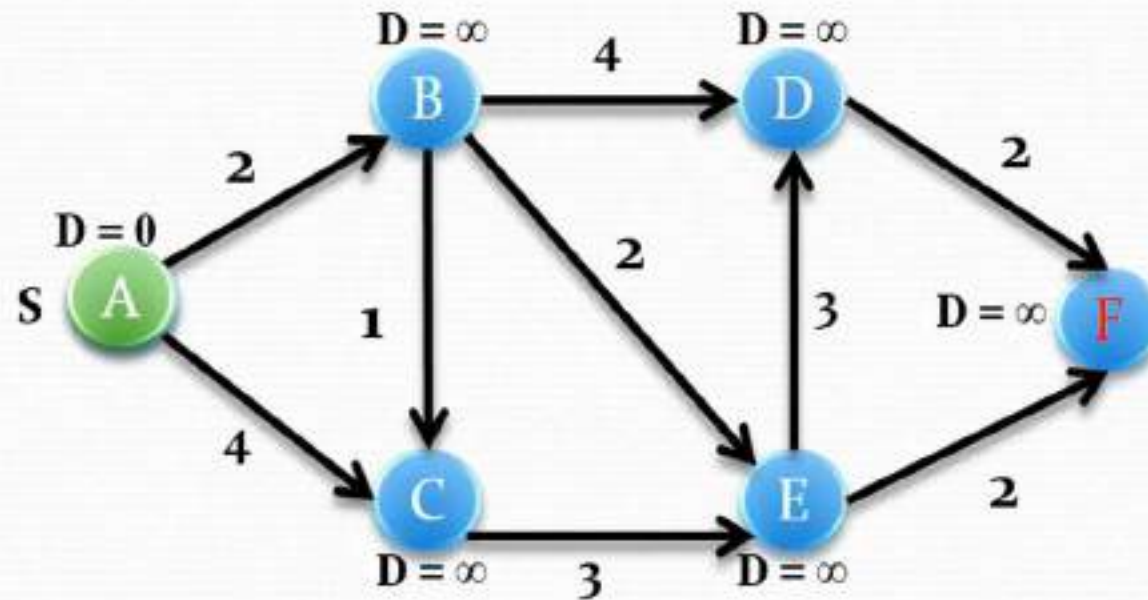
Dijkstra's Rules

Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.



Q ≤ V	A	B	C	D	E	F
A						

Dijkstra's Rules

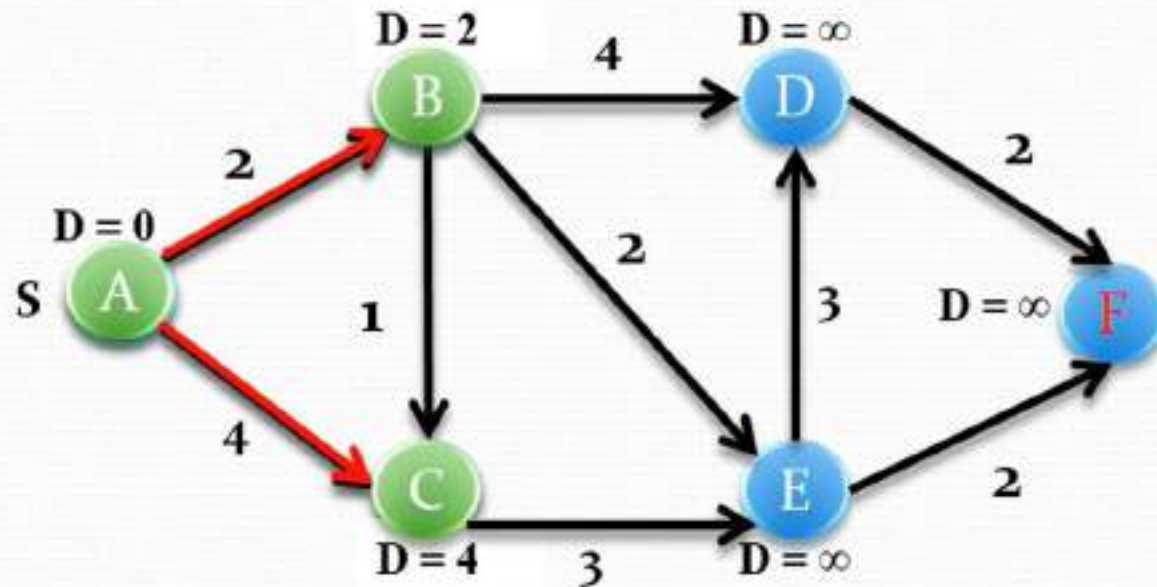
Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.

If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}

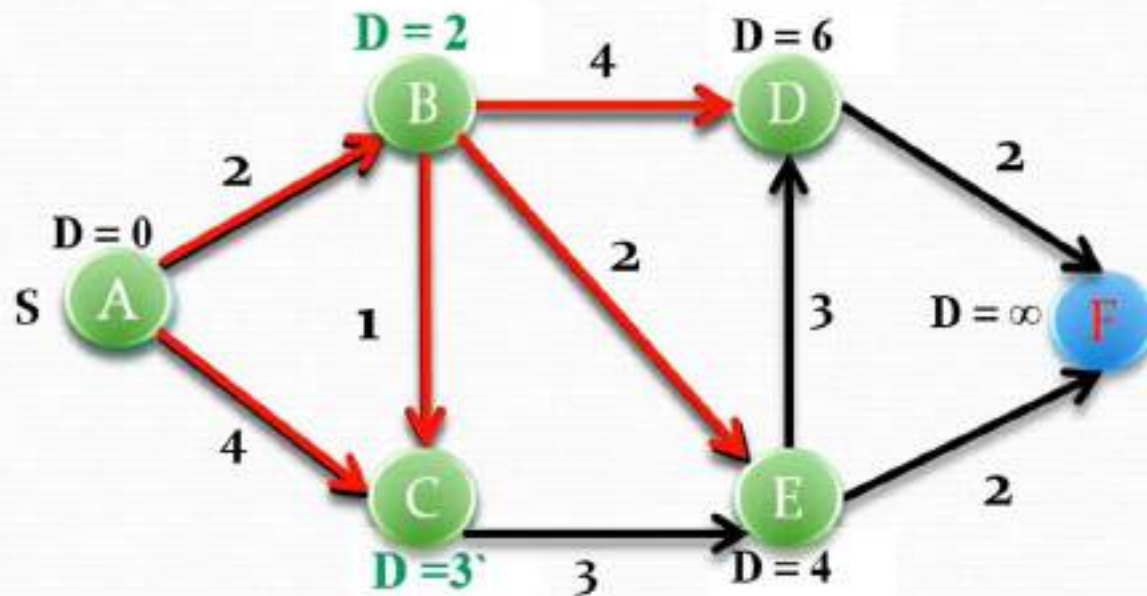


Q ≤ V	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞ ^A	∞ ^A	∞ ^A

Dijkstra's Rules

- Rule 1:** Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.
- Rule 2:** Relax all vertices adjacent to the current vertex.
- Rule 3:** Choose the closest vertex as next current vertex.
- Rule 4:** Repeat Rule2 and Rule 3 until the queue or reach the destination.

If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}



Q ≤ V	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞ ^A	∞ ^A	∞ ^A
B	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A

Dijkstra's Rules

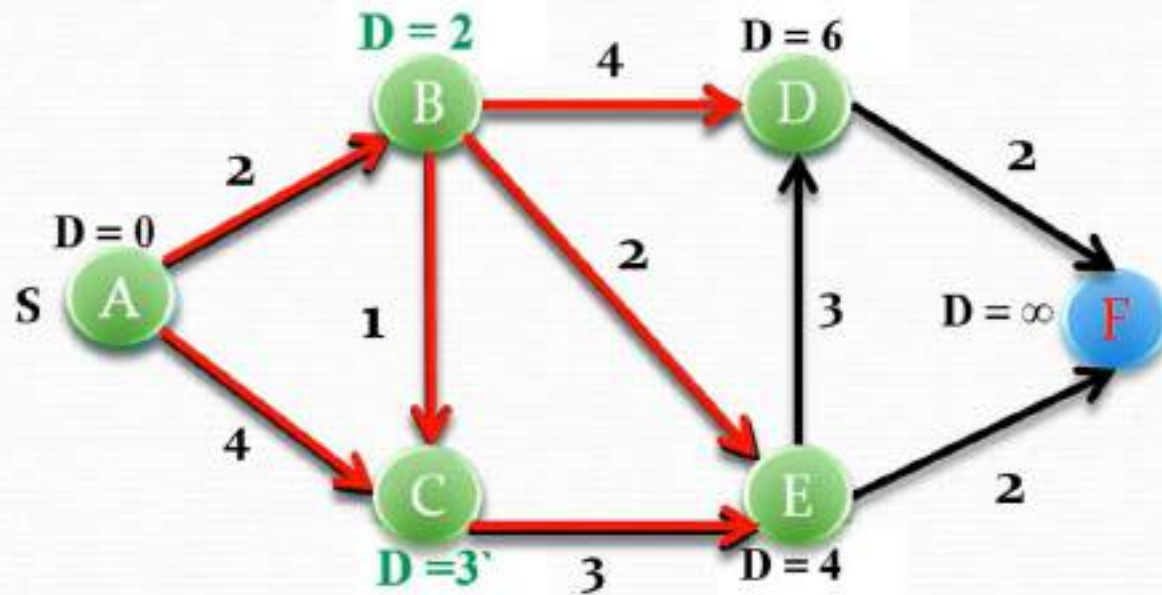
Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.

If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}



Q ≤ V	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞ ^A	∞ ^A	∞ ^A
B	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A
C	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A

Dijkstra's Rules

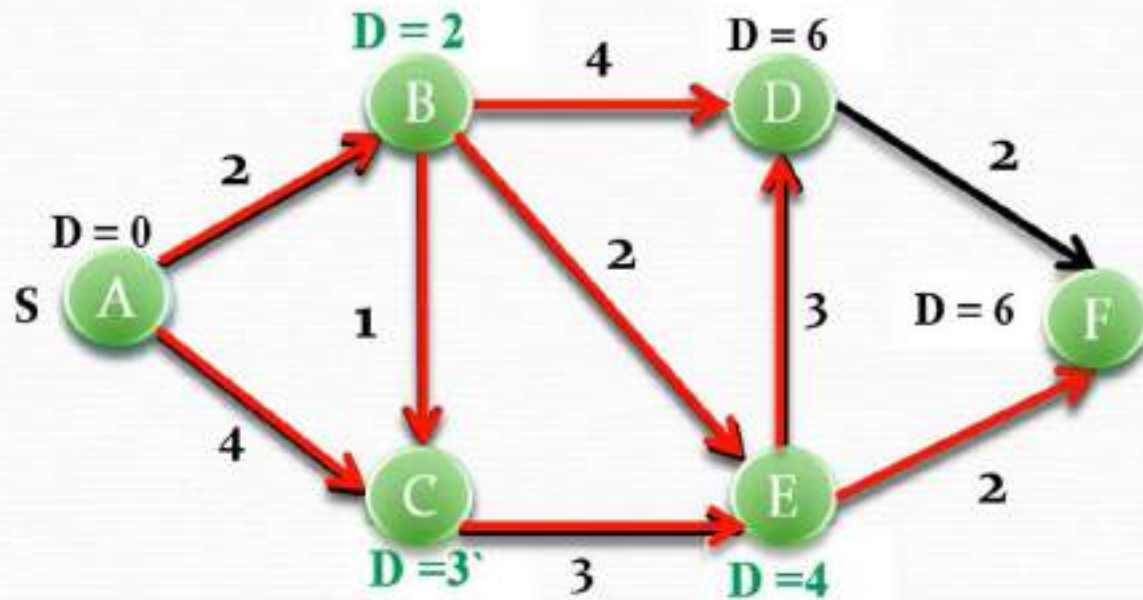
Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.

If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}



$Q \leq V$	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞^A	∞^A	∞^A
B	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞^A
C	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞^A
E	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E

Dijkstra's Rules

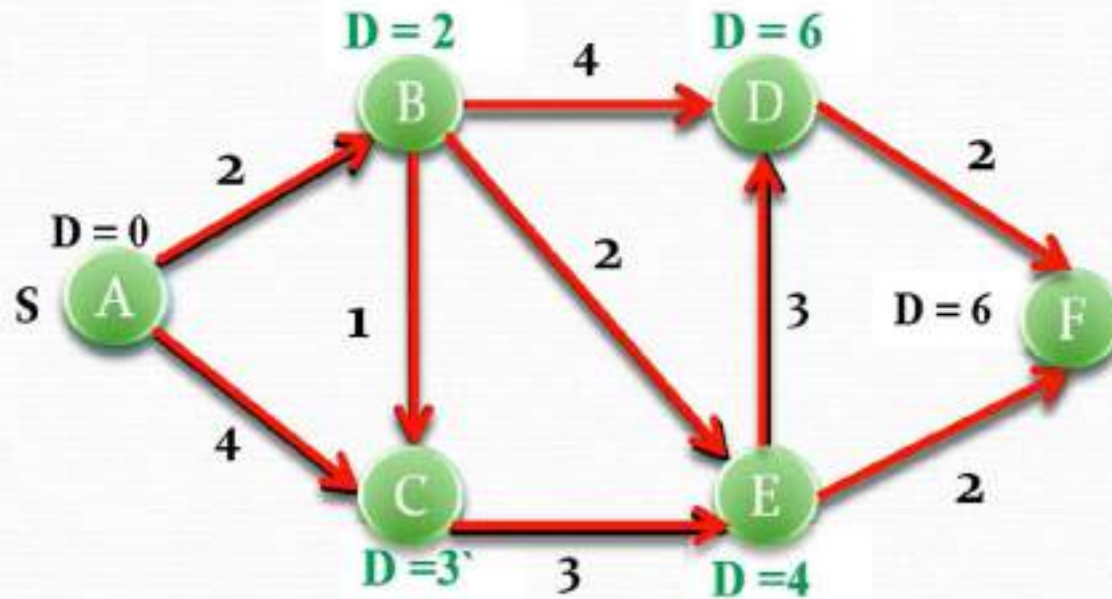
Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.

If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}



Q ≤ V	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞ ^A	∞ ^A	∞ ^A
B	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A
C	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A
E	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E
D	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E

Dijkstra's Rules

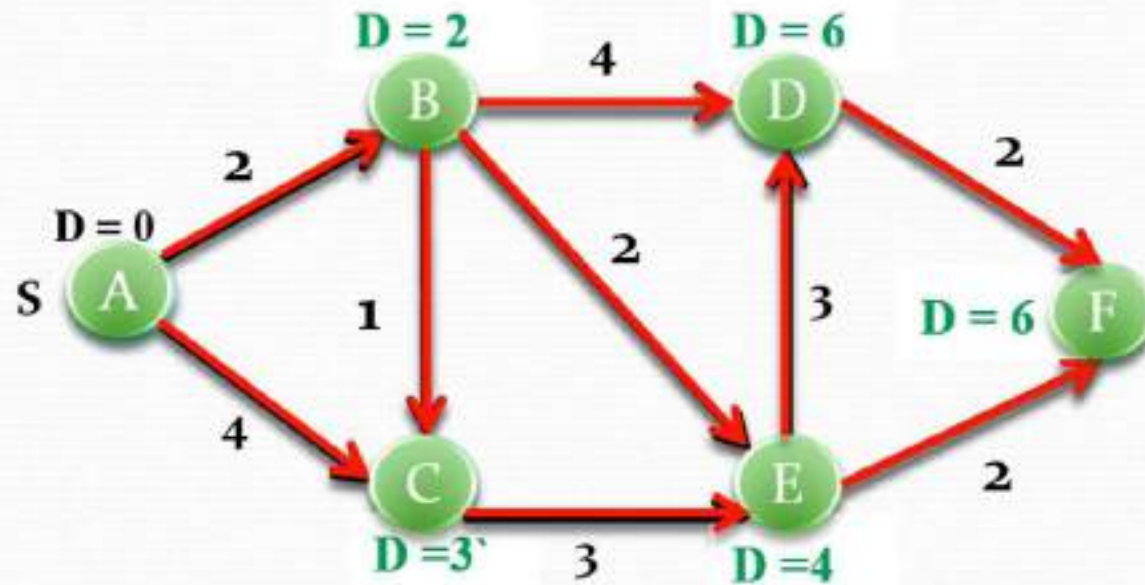
Rule 1: Make sure there is no negative edges. Set distance to source vertex as zero and set all other distances to infinity.

Rule 2: Relax all vertices adjacent to the current vertex.

Rule 3: Choose the closest vertex as next current vertex.

Rule 4: Repeat Rule2 and Rule 3 until the queue or reach the destination.

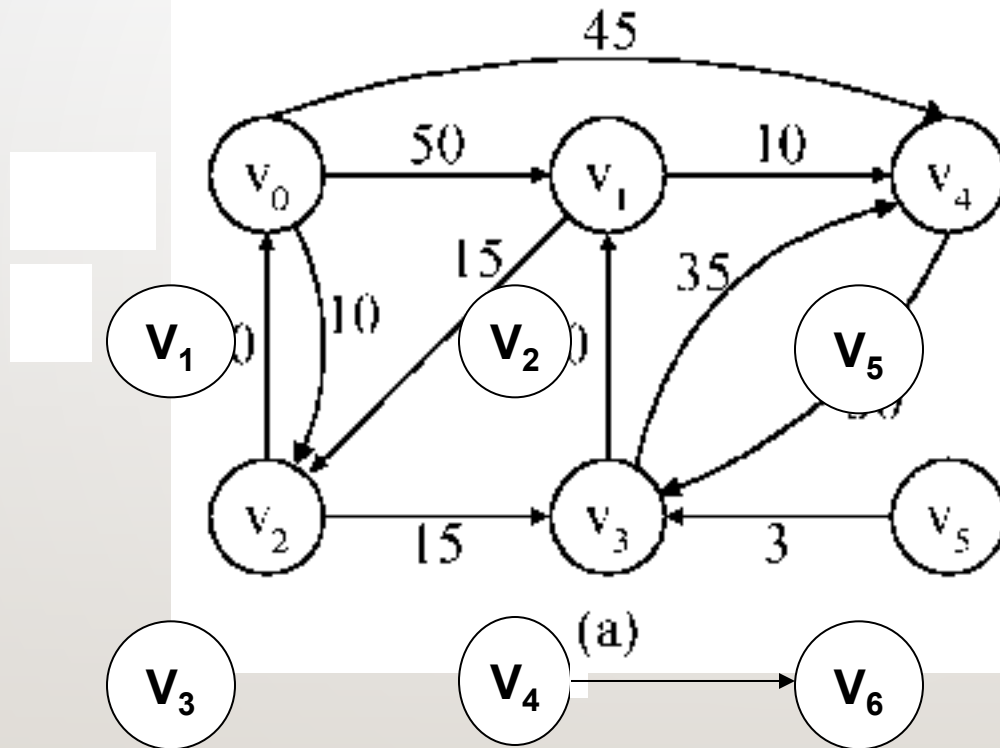
If $(D[C] + D[AdjEdge]) < D[Adj]$ {Update Adj's D with new shortest path}



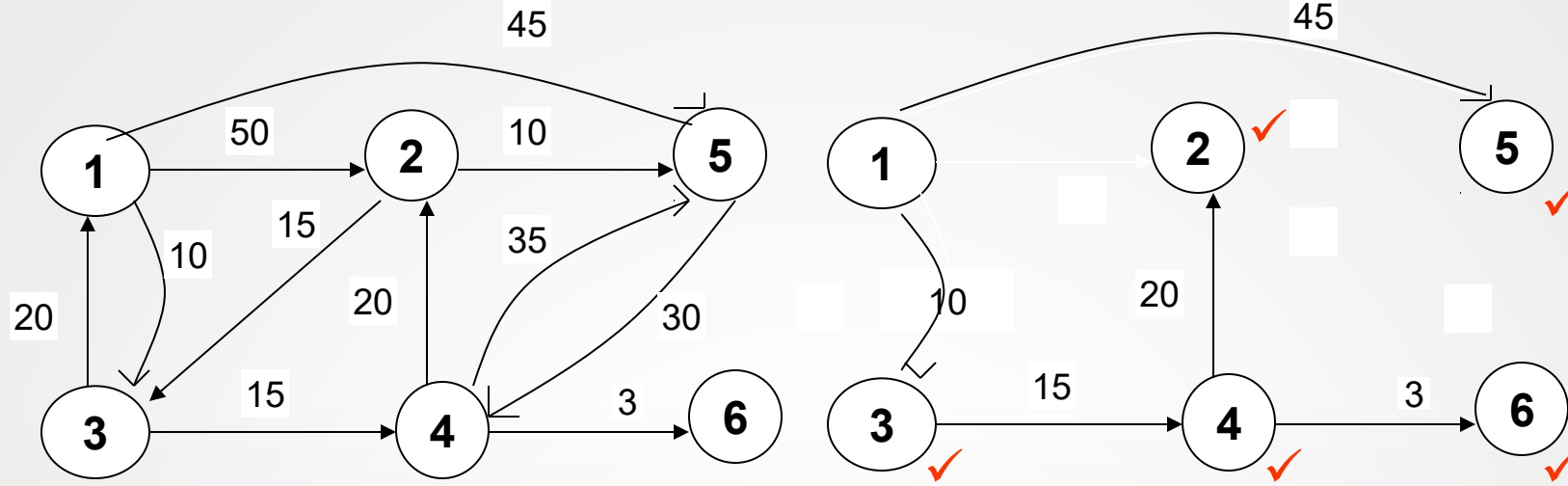
Q ≤ V	A	B	C	D	E	F
A	0 ^A	2 ^A	4 ^A	∞ ^A	∞ ^A	∞ ^A
B	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A
C	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	∞ ^A
E	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E
D	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E
F	0 ^A	2 ^A	3 ^B	6 ^B	4 ^B	6 ^E

THE SINGLE-SOURCE SHORTEST PATH PROBLEM (SSSP)

- Given a positively weighted directed graph G with a source vertex v_1 , find the shortest paths from v_1 to all other vertices in the graph



	Path	Length
1)	$v_0 v_2$	10
2)	$v_0 v_2 v_3$	25
3)	$v_1 v_3 v_2 v_3 v_1$	45
4)	$v_1 v_3 v_4 v_4$	45
	$v_1 v_3 v_4 v_2$	
	$v_1 v_5$	(b)
5)	$v_1 v_3 v_4 v_6$	28



Visited Nodes	A	B	C	D	E	F	PATH	DISTANCE
A	0	∞	∞	∞	∞	∞	----	0
C		50	10	∞	45	∞	AC	10
D		50		25	45	∞	ACD	25
F		45			45	28	ACDF	28
B		45			45		ACDB	45
E					45		AE	45

SSSP-DIJKSTRA'S ALGORITHM

- Dijkstra's algorithm assumes that $\text{cost}(e) \geq 0$ for each e in the graph.
- Maintains a set S of vertices whose SP from v (source) has been determined.
- a) Select the next minimum distance node u , which is not in S .
- b) *for* each node w adjacent to u *do*
 if($\text{dist}[w] > \text{dist}[u] + \text{cost}[u, w]$) then
 $\text{dist}[w] := \text{dist}[u] + \text{cost}[u, w]$;
- Repeat step (a) and (b) until $S = n$ (number of vertices).

GREEDY ALGORITHM TO GENERATE SHORTEST PATHS

1. Algorithm **ShortestPaths**(v, cost, dist, n)
2. //dist[j], $1 \leq j \leq n$, is set to the length of the shortest path
3. //from vertex v to
4. // vertex j in a digraph G with n vertices. dist[v] is set to zero.
5. // G is represented by its cost adjacency matrix cost[1:n, 1:n].
6. {
7. for i := 1 to n do
8. { // Initialize S.
9. s[i] := false; dist[i] := cost[v,i]
10. }
11. s[v] := true; dist[v] := 0.0 ;

Inputs:

- **v:** The source vertex from which shortest paths are calculated.
- **cost:** The adjacency matrix representing the graph. `cost[i][j]` is the weight of the edge from vertex i to vertex j . If there's no direct edge, it's typically set to infinity.
- **dist:** An array where `dist[i]` will store the minimum distance from the source vertex v to vertex i .
- **n:** The total number of vertices in the graph.

Outputs:

- **dist:** The `dist` array will contain the shortest path distances from the source vertex v to every other vertex in the graph.

Algorithm Steps:

1. Initialization (Lines 6-8):

- $s[i]$: A boolean array where each element indicates whether the vertex i has been included in the set S , which stores vertices for which the shortest path has been found.
- For each vertex i , initialize $s[i]$ to `false` (indicating that no vertices are initially in the set S), and set $dist[i]$ to $cost[v][i]$, which is the direct distance from the source vertex v to vertex i .
- Set $s[v]$ to `true` because the shortest path to the source itself is zero ($dist[v] := 0.0$).

```
12. for num: = 2 to n do
13. {
14. // Determine n-1 paths from v.
15. Choose u from among those vertices not in S
16. such that dist[u] is minimum;
17. s[u] := true;           // Put u in S.
18. for ( each w adjacent to u with S[w]= false) do
19. // Update distance
20. if( dist[w] > dist[u] + cost[u, w]) ) then
21. dist[w] := dist[u] + cost[u, w];
22. }
23. }
```

2. Main Loop (Lines 12-22):

- The algorithm proceeds with a loop that runs $n - 1$ times, corresponding to the number of paths to determine (one for each vertex except the source).
- **Choosing the next vertex (Lines 15-17):**
 - Select a vertex u that is not yet in set S and has the minimum distance ($\text{dist}[u]$) from the source v among all vertices not yet in S .
 - Add u to S ($s[u] := \text{true}$).
- **Updating the distances (Lines 18-21):**
 - For each vertex w that is adjacent to u and not yet in set S ($s[w] = \text{false}$), check if a shorter path can be found through u . This is done by comparing $\text{dist}[w]$ with $\text{dist}[u] + \text{cost}[u][w]$.
 - If $\text{dist}[w] > \text{dist}[u] + \text{cost}[u][w]$, update $\text{dist}[w]$ to the new shorter distance ($\text{dist}[w] := \text{dist}[u] + \text{cost}[u][w]$).

3. Final Output:

- After the loop finishes, the `dist` array will contain the shortest path distances from the source vertex v to every other vertex in the graph.

Example Walkthrough:

Consider a graph with vertices 1, 2, 3, 4 and source vertex $v = 1$. Assume the adjacency matrix `cost` is given. Initially, `dist[i]` is set to the direct cost from vertex 1 to vertex i . Then, at each step of the loop, the algorithm finds the closest vertex not yet in the set S , adds it to S , and updates the distances for its adjacent vertices.

TIME COMPLEXITY OF DIJKSTRA'S ALGORITHM

- The for loop of line 7 takes $o(n)$.
- The for loop of line 12 takes $o(n)$.
 - Each execution of this loop requires $o(n)$ time at lines 15 and 18.
 - So the total time for this loop is $o(n^2)$.
- Therefore, total time taken by this algorithm is $o(n^2)$.

Complexity:

- The time complexity of Dijkstra's algorithm depends on the data structures used:
 - With a simple array for the priority queue, it runs in $O(n^2)$.
 - With a min-heap or priority queue, it can run in $O((n + m) \log n)$, where m is the number of edges.

DIJKSTRA'S ALGORITHM PQ		
Best	Average	Worst
$O((V + E) \cdot \log V)$	<i>same</i>	<i>same</i>



Weighted
Directed
Graph



Priority
queue



Array



Overflow

singleSourceShortest (G, s)

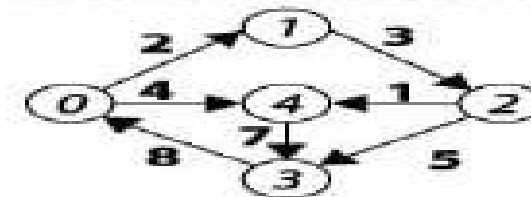
1. $PQ = \text{new Priority Queue}$
2. **foreach** $v \in V$ **do**
3. $\text{dist}[v] = \infty$
4. $\text{pred}[v] = -1$
5. $\text{dist}[s] = 0$
6. **foreach** $v \in V$ **do**
7. insert ($v, \text{dist}[v]$) into PQ
8. **while** (PQ is not empty) **do**
9. $u = \text{getMin}(PQ)$
10. **foreach** neighbor v of u **do**
11. $w = \text{weight of edge } (u,v)$
12. $\text{newLen} = \text{dist}[u] + w$
13. **if** ($\text{newLen} < \text{dist}[v]$) **then**
14. decreaseKey (PQ, v, newLen)
15. $\text{dist}[v] = \text{newLen}$
16. $\text{pred}[v] = u$
- end**

5th iteration: remove 3 and done



0	1	2	3	4
0	2	5	10	4

Create PQ from neighbors v of vertex $s = 0$ based on $\text{dist}[v]$



0	1	2	3	4
0	∞	∞	∞	∞

dist[v]

Remove vertex u from PQ with least distance from s . If path from (s,u) and (u,v) is shorter than best computed distance (s,v) , adjust $\text{dist}[v]$ and PQ.

1st iteration: remove 0 and adjust



0	1	2	3	4
0	2	∞	∞	4

$$(0,0) + (0,1) < (0,1)$$

$$(0,0) + (0,4) < (0,4)$$

2nd iteration: remove 1 and adjust



0	1	2	3	4
0	2	5	∞	4

$$(0,1) + (1,2) < (0,2)$$

3rd iteration: remove 4 and adjust



0	1	2	3	4
0	2	5	11	4

$$(0,4) + (4,3) < (0,3)$$

4th iteration: remove 2 and adjust

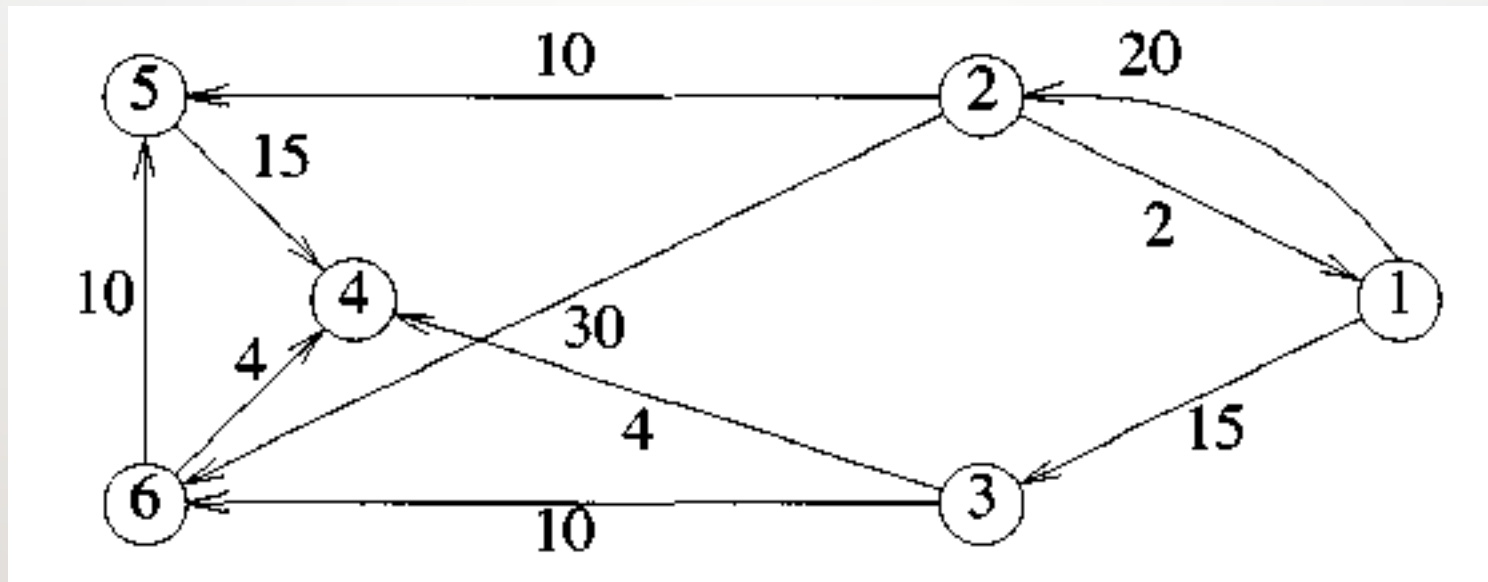


0	1	2	3	4
0	2	5	10	4

$$(0,2) + (2,3) < (0,3)$$

Example:

Use shortest path algorithm to obtain in nondecreasing order of the lengths of the shortest paths from vertex 1 to all remaining vertices.



SAMPLE QUESTIONS

- Explain about Dijkstra's
- Analyse the time complexity of single source shortest path
- What is the term Relaxation in Dijkstra's
- Explain the concept of finding the shortest path from a single source vertex to all other vertices in a weighted graph

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

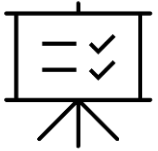
All Shortest Path Problem

AIM OF THE SESSION



To familiarize students with the concept of All pairs Shortest Path Problem

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- All Pairs Shortest Path Problem.
2. Describe :- Solve All pairs shortest path problem using Floyd-Warshall Algorithm .

LEARNING OUTCOMES



At the end of this session, you should be able to:

1. Define :- All Pairs Shortest path Problem.
2. Describe :- solve All Pairs shortest path problem using Floyd-Warshall Algorithm
3. Summarize:- Finding the shortest paths from all source vertices to all other vertices in a weighted graph.

Floyd-Warshall Algorithm

Floyd-Warshall Algorithm is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph. This algorithm works for both the directed and undirected weighted graphs. But, it does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative).

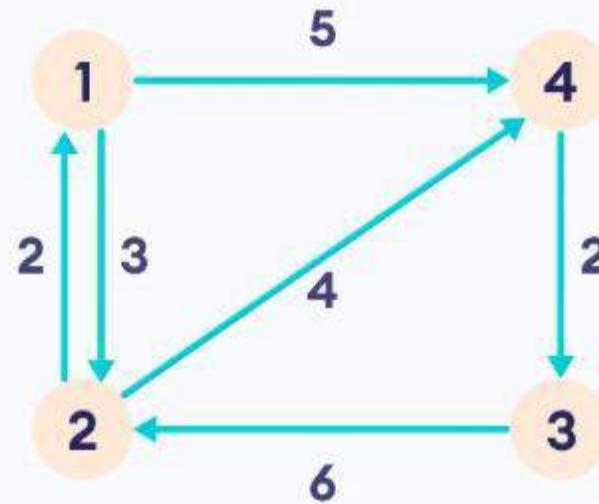
A weighted graph is a graph in which each edge has a numerical value associated with it.

Floyd-Warshall algorithm is also called as Floyd's algorithm, Roy-Floyd algorithm, Roy-Warshall algorithm, or WFI algorithm.

This algorithm follows the **dynamic programming** approach to find the shortest paths.

How Floyd-Warshall Algorithm Works?

Let the given graph be:



Initial Graph

Follow the steps below to find the shortest path between all the pairs of vertices.

1. Create a matrix A^0 of dimension $n \times n$ where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph.

Each cell $A[i][j]$ is filled with the distance from the i^{th} vertex to the j^{th} vertex. If there is no path from i^{th} vertex to j^{th} vertex, the cell is left as infinity.

A representation: a weight matrix where

$W(i,j)=0$ if $i=j$.

$W(i,j)=\infty$ if there is no edge between i and j .

$W(i,j)$ ="weight of edge"

$$A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Fill each cell with the distance between i^{th} and j^{th} vertex

2. Now, create a matrix A^1 using matrix A^0 . The elements in the first column and the first row are left as they are. The remaining cells are filled in the following way.

Let k be the intermediate vertex in the shortest path from source to destination. In this step, k is the first vertex. $A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$.

That is, if the direct distance from the source to the destination is greater than the path through the vertex k , then the cell is filled with $A[i][k] + A[k][j]$.

In this step, k is vertex 1. We calculate the distance from source vertex to destination vertex through this vertex k .

$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through this vertex k

$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through this vertex k

For example: For $A^1[2, 4]$, the direct distance from vertex 2 to 4 is 4 and the sum of the distance from vertex 2 to 4 through vertex 1 (ie. from vertex 2 to 1 and from vertex 1 to 4) is 7. Since $4 < 7$, $A^0[2, 4]$ is filled with 4.

3. Similarly, A^2 is created using A^1 . The elements in the second column and the second row are left as they are.

In this step, k is the second vertex (i.e. vertex 2). The remaining steps are the same as in **step 2**.

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & \infty & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through this vertex 2

4. Similarly, A^3 and A^4 is also created.

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & \infty & \\ & 0 & \infty & \\ 3 & 1 & 0 & 5 \\ & & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 2 & 0 & 4 \\ 3 & 3 & 1 & 0 & 5 \\ 4 & 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through this vertex 3

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 2 & 0 & 4 \\ 3 & 3 & 1 & 0 & 5 \\ 4 & 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through this vertex 4

5. A^4 gives the shortest path between each pair of vertices.
-

Floyd-Warshall Algorithm

```
n = no of vertices
A = matrix of dimension n*n
for k = 1 to n
    for i = 1 to n
        for j = 1 to n
             $A^k[i, j] = \min (A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$ 
return A
```

Floyd Warshall Algorithm Complexity

Time Complexity

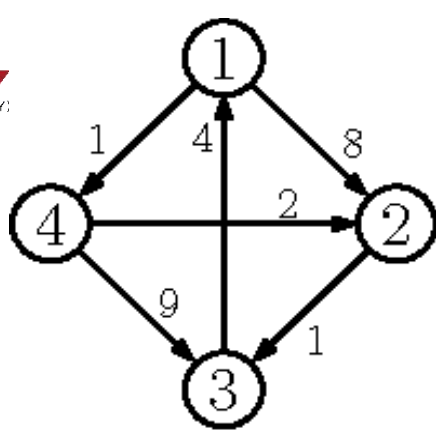
There are three loops. Each loop has constant complexities. So, the time complexity of the Floyd-Warshall algorithm is $O(n^3)$.

Space Complexity

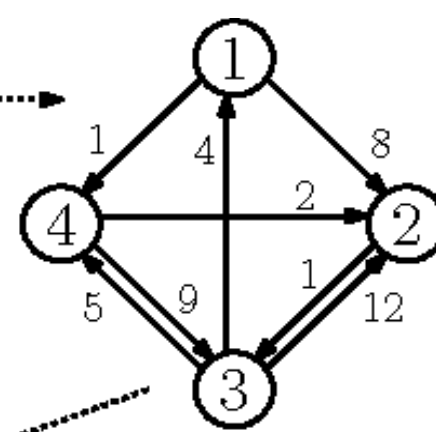
The space complexity of the Floyd-Warshall algorithm is $O(n^2)$.

Floyd Warshall Algorithm Applications

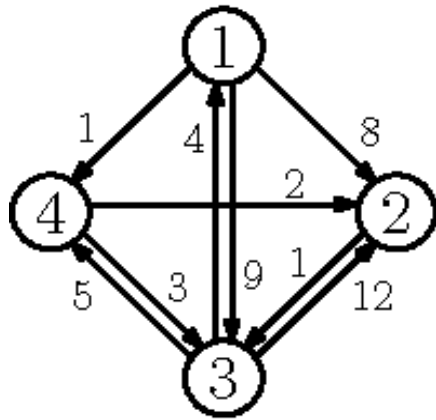
- To find the shortest path in a directed graph
- To find the transitive closure of directed graphs
- To find the Inversion of real matrices
- For testing whether an undirected graph is bipartite



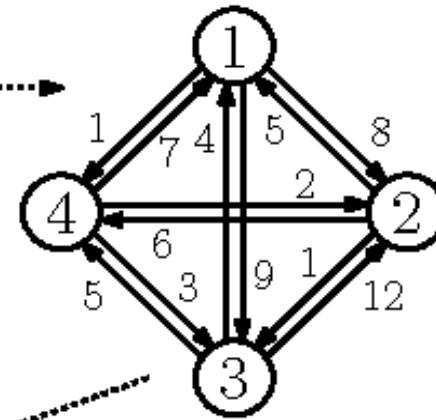
$$d^{(0)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix}$$



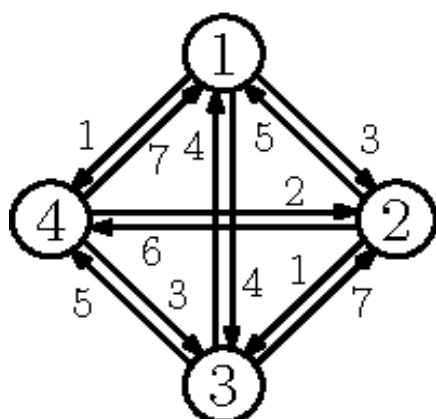
$$d^{(1)} = \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix}$$



$$d^{(2)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix}$$



$$d^{(3)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$



$$d^{(4)} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

$$\text{final} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

OR

All-Pairs Shortest Paths – Floyd-Warshall Algorithm

- The Floyd-Warshall Algorithm is used to find the shortest path between all pairs of nodes in a weighted graph.
- This algorithm can support the both positive and negative weights of the graph.
- It can solving a wide range of network and connectivity problems.

All-Pairs Shortest Paths – Floyd-Warshall Algorithm

- The Floyd-Warshall Algorithm is an all pairs shortest path algorithm unlike Dijkstra and Bellman Fold, which are single source shortest algorithms.
- These algorithms can work for both directed and undirected graphs but not for negative edges of the graph.
- Floyd Warshall can follow the dynamic programming approach to check every possible path going via every possible node in order to calculate distance between every pair of nodes.

Floyd-Warshall Algorithm

- Suppose we have a **Graph $G[][]$** with **V** vertices from **1 to N** .
- Now we have to evaluate a **shortestPathMatrix $[][]$** where shortestPathMatrix[i][j] represents the shortest path between vertices i and j.
- Obviously the shortest path between i to j will have some k number of intermediate nodes.
- The idea behind floyd warshall algorithm is to treat each and every vertex from 1 to N as an intermediate node one by one.

Floyd-Warshall Algorithm

Step 1: Initialize the solution matrix same as the input graph matrix.

Step 2: Update the solution matrix by considering all vertices as an intermediate vertex.

Step 3: Pick all vertices one by one and updates all shortest paths which include the picked vertex as an intermediate vertex in the shortest path.

Step 4: The picked vertex number k as an intermediate vertex, we already have considered vertices $\{0, 1, 2, \dots, k-1\}$ as intermediate vertices.

Floyd-Warshall Algorithm

Step 5: For every pair (i, j) of the source and destination vertices respectively, there are two possible cases.

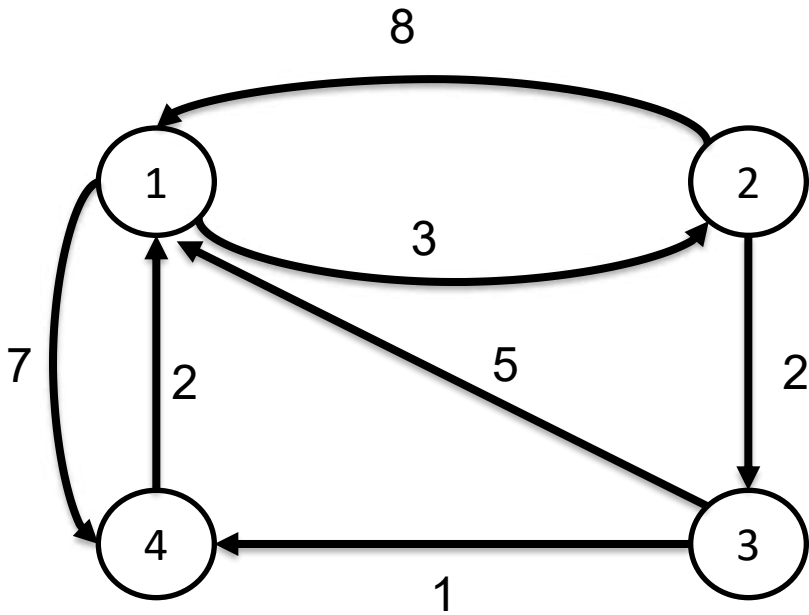
Step 5.1: k is not an intermediate vertex in shortest path from i to j.

We keep the value of $\text{dist}[i][j]$ as it is.

Step 5.2: k is an intermediate vertex in shortest path from i to j.

We update the value of $\text{dist}[i][j]$ as $\text{dist}[i][k] + \text{dist}[k][j]$, if $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$

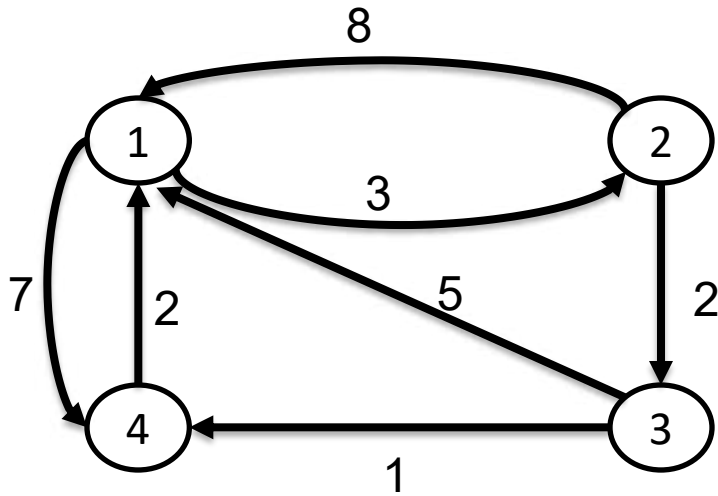
Floyd-Warshall Algorithm - Example



Suppose we have a graph as show in here

Floyd-Warshall Algorithm - Example

Initialize the $Distance[][]$ matrix using the input graph such that
 $Distance[i][j]$ = weight of edge from i to j , also
 $Distance[i][j] = \infty$ if there is no edge from i to j .



$A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

Floyd-Warshall Algorithm - Example

- Select the intermediate node as 1
- 1st row and 1st column are same as A^0
- Same node distance is 0
- The rest of the cells can be calculated as below formula.

$$A^k[i, j] = \min\{A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j]\}$$

Where k is an intermediate vertex.

 $A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

 $A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0		
3	5		0	
4	2			0

Floyd-Warshall Algorithm - Example

$$A^1[2, 3] = \min\{A^0[2, 3], A^0[2, 1] + A^0[1, 3]\}$$

$$A^1[2, 3] = \min\{2, 8 + \infty\}$$

$$A^1[2, 3] = 2$$

$$A^1[2, 4] = \min\{A^0[2, 4], A^0[2, 1] + A^0[1, 4]\}$$

$$A^1[2, 4] = \min\{\infty, 8 + 7\}$$

$$A^1[2, 4] = 15$$

$A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5		0	
4	2			0

Floyd-Warshall Algorithm - Example

$$A^1[3, 2] = \min\{A^0[3, 2], A^0[3, 1] + A^0[1, 2]\}$$

$$A^1[3, 2] = \min\{\infty, 5 + 3\}$$

$$A^1[3, 2] = 8$$

$$A^1[3, 4] = \min\{A^0[3, 4], A^0[3, 1] + A^0[1, 4]\}$$

$$A^1[3, 4] = \min\{1, 5 + 4\}$$

$$A^1[3, 4] = 1$$

$A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2			0

Floyd-Warshall Algorithm - Example

$$A^1[4, 2] = \min\{A^0[4, 2], A^0[4, 1] + A^0[1, 2]\}$$

$$A^1[4, 2] = \min\{\infty, 2 + 3\}$$

$$A^1[4, 2] = 5$$

$$A^1[4, 3] = \min\{A^0[4, 3], A^0[4, 1] + A^0[1, 3]\}$$

$$A^1[4, 3] = \min\{\infty, 2 + \infty\}$$

$$A^1[4, 3] = \infty$$

$A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

Floyd-Warshall Algorithm - Example

$A^0 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

$A^2 =$

	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

$A^3 =$

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

Floyd-Warshall Algorithm - Example

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	5	8	0	1
4	2	5	∞	0

$A^2 =$

	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

$A^3 =$

	1	2	3	4
1	0	3	5	6
2	7	0	2	3
3	5	8	0	1
4	2	5	7	0

$A^4 =$

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

Floyd-Warshall Algorithm - Example

$A^4 =$

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

- Since all the nodes have been treated as an intermediate node, we can now return the updated distance matrix $A[i][j]$ is as our answer matrix.

Floyd-Warshall Algorithm - Example

- Dijkstra's algorithm can run n^2 time for single source vertices, for n vertices it will take $n^2 \times n = n^3$ time is required.
- The pseudo code for the Floyd-warshall algorithm is:

```
for k = 1 to n
  for i = 1 to n
    for j = 1 to n
       $A^k[i, j] = \min\{A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j]\}$ 
```

There fore the time complexity is $O(n^3)$

- **Objective:** Find the shortest paths from all source vertices to all other vertices in a weighted graph.

- **Approach:**

- Initialize the solution matrix same as the input graph matrix as a first step.
- Then update the solution matrix by considering all vertices as an intermediate vertex.
- The idea is to pick all vertices one by one and updates all shortest paths which include the picked vertex as an intermediate vertex in the shortest path.
- When we pick vertex number **k** as an intermediate vertex, we already have considered vertices **{0, 1, 2, .. k-1}** as intermediate vertices.

- **Algorithmic Solutions:** Algorithms like Floyd Warshall algorithm used to solve this problem efficiently.

SELF-ASSESSMENT QUESTIONS

The Floyd-Warshall algorithm is based on which programming paradigm?

- (a) Divide and conquer
- (b) Greedy method
- (c) **Dynamic programming**
- (d) Backtracking

What is the time complexity of the Floyd-Warshall algorithm for a graph with n vertices?

- (a) **$O(n^3)$**
- (b) $O(n^2)$
- (c) $O(n^4)$
- (d) $O(n \log n)$

TERMINAL QUESTIONS

1. Given the adjacency matrix of a graph:

$$\begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$$

Use the Floyd-Warshall algorithm to compute the shortest path matrix. Show all intermediate steps.

2. Describe the differences between the Floyd-Warshall algorithm and Dijkstra's algorithm for solving shortest path problems.

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU

Advanced Algorithms & Data Structures



Department of CSE

ADVANCED ALGORITHMS AND DATA STRUCTURES 23CS03HF

Topic:

Minimum Cost Spanning Tree

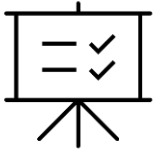
Session - 25

AIM OF THE SESSION



To familiarize students with the concept of Spanning Tree

INSTRUCTIONAL OBJECTIVES



This Session is designed to:

1. Demonstrate :- Minimum Cost Spanning Tree.
2. Describe :- Minimum spanning tree and its related algorithms like Prim's and Kruskal's methods

LEARNING OUTCOMES



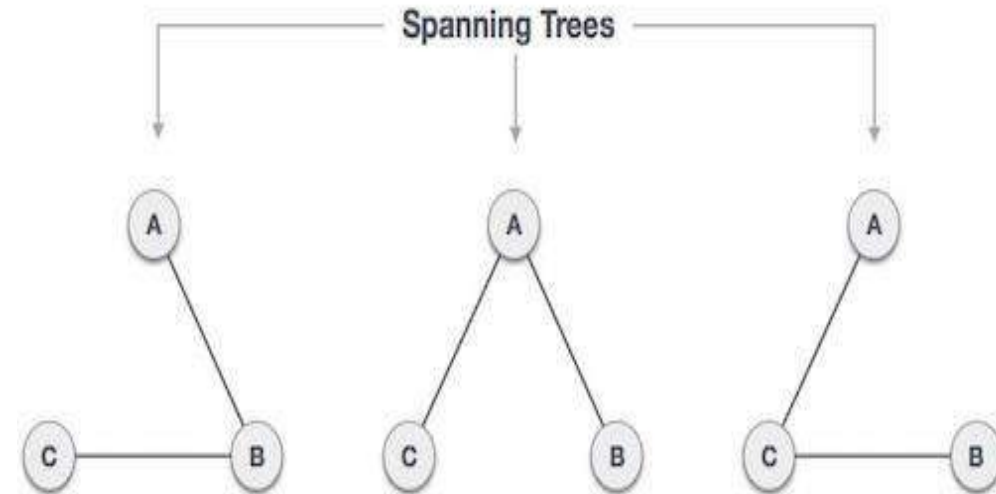
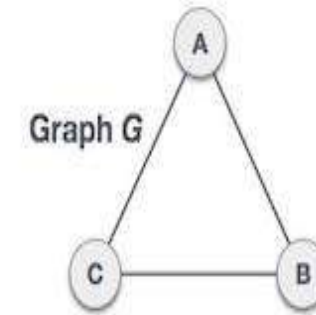
At the end of this session, you should be able to:

1. Define :- Minimum Cost Spanning Tree.
2. Describe :- Minimum spanning tree and its related algorithms like Prim's and Kruskal's methods
3. Summarize:- How to build an MST for a given graph using Prim's and Kruskal's Algorithm.

- A **spanning tree** is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph.

(Or)

- It is a subset of the edges of the graph that forms a tree (acyclic) where every node of the graph is a part of the tree.

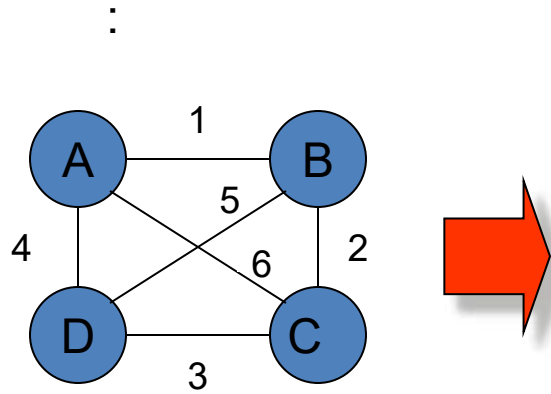


Properties of a Spanning Tree

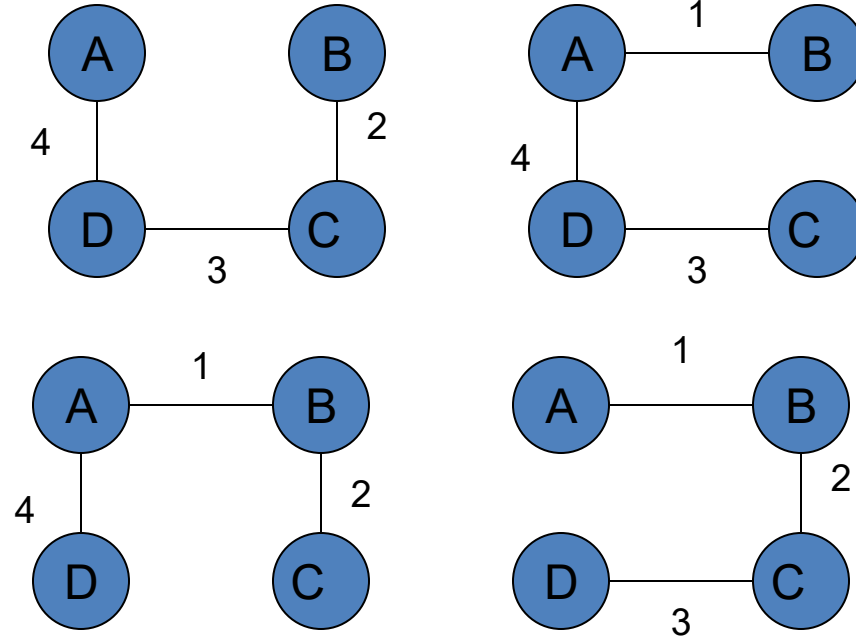
- The number of **vertices (V)** in the graph and the spanning tree is **the same**.
- There is a fixed number of edges in the spanning tree which is equal to one less than the total number of vertices (**$E = V-1$**).
- The spanning tree **should not be disconnected**, as in there should only be a single source of component, not more than that.
- The spanning tree should be **acyclic**, which means there would not be any cycle in the tree.
- The total cost (or weight) of the spanning tree is defined as the sum of the edge weights of all the edges of the spanning tree.
- There can be many possible spanning trees for a graph.

Properties of a Spanning Tree:

- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.
- A complete undirected graph can have an **n^{n-2} number of spanning trees**.
- Every connected and undirected graph contains at least one spanning tree.
- The disconnected graph does not have any spanning tree.
- In a complete graph, we can remove maximum $(e-n+1)$ edges to construct a spanning tree.



Undirected Graph

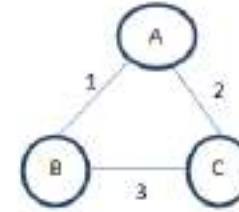


...

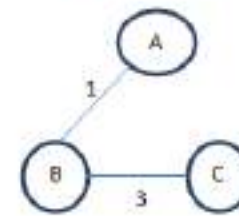
Some Spanning Trees

A Minimum Spanning Tree (MST) is defined as a spanning tree that has the minimum weight among all the possible spanning trees.

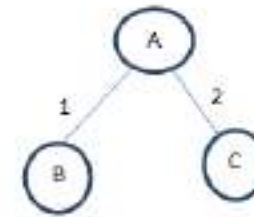
- The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph.



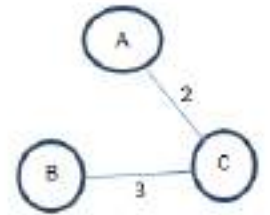
The resultant spanning trees are shown below-



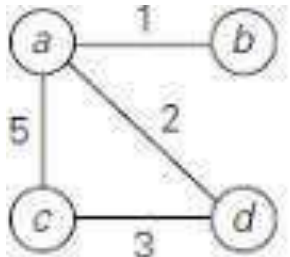
Spanning cost = $1 + 3 = 4$



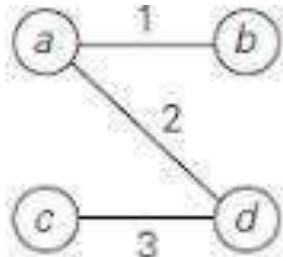
Spanning cost = $1 + 2 = 3$



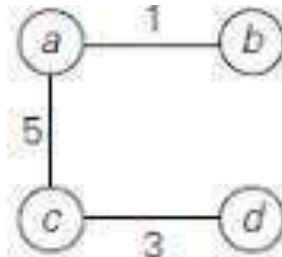
Spanning cost = $2 + 3 = 5$



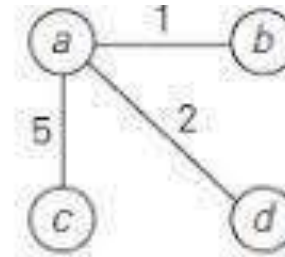
graph



$w(T_1) = 6$



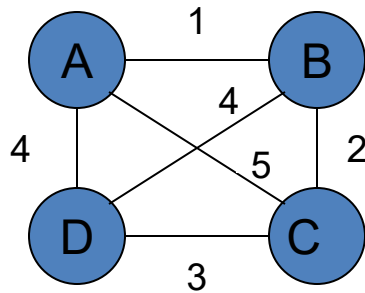
$w(T_2) = 9$



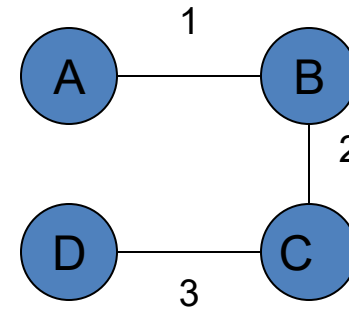
$w(T_3) = 8$

Minimum Cost Spanning Tree (MST)

- A **minimum spanning tree** is the one among all the spanning trees with the **smallest total cost**.



Undirected Graph



Minimum Spanning Tree

Applications of MSTs

- Computer Networks
 - How to connect a set of computers using the **minimum amount of wire..**
- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis

Kruskal's Method

- Start with a forest that has **no edges**.
- **Add** the next minimum cost edge to the forest if it will **not** cause a cycle.
- Continue this process until the tree has **$n - 1$** edges.

Kruskal's Algorithm Overview

Kruskal's algorithm is a greedy algorithm used to find the **Minimum Spanning Tree (MST)** of a graph. The algorithm works as follows:

1. **Sort all edges** of the graph in non-decreasing order of their weights.
2. **Pick the smallest edge** and check if it forms a cycle with the spanning tree formed so far. If it doesn't, add it to the MST.
3. **Repeat until you have $V - 1$ edges** in the MST, where V is the number of vertices in the graph.

To efficiently pick the smallest edge, instead of sorting all edges first, a **min-heap** can be used to extract the smallest edge at each step.

- Often uses **Union-Find (Disjoint Set Union)** to detect cycles efficiently.

In Kruskal's algorithm, a **min-heap** is used to manage and efficiently retrieve the smallest edge during the construction of the MST.

1. **Insert all edges into the min-heap:** Initially, all edges of the graph are inserted into a min-heap. Since a min-heap provides quick access to the smallest element, this allows for efficient extraction of the smallest edge.
2. **Extract the smallest edge:** During each iteration of the algorithm, the edge with the minimum weight is extracted from the min-heap.
3. **Check for cycles:** The extracted edge is checked to ensure that it doesn't form a cycle with the edges that are already in the MST. This is done using the **Union-Find (Disjoint Set Union)** data structure.
4. **Add to the MST:** If no cycle is formed, the edge is added to the MST. This process continues until the MST contains $V - 1$ edges.

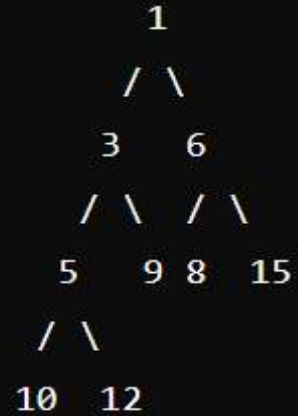
A **min-heap** is a special kind of binary tree data structure that satisfies the **heap property**, where the key at the root must be the minimum among all keys in the binary heap. This property is recursively true for all sub-trees in the binary heap. In simpler terms, in a min-heap, the value of each parent node is less than or equal to the values of its children.

Properties of Min-Heap:

1. **Binary Tree:** A min-heap is a complete binary tree, meaning all levels are fully filled except possibly the last level, which is filled from left to right.
2. **Heap Property:** For any node i , the value of the node is less than or equal to the values of its children. This ensures that the smallest value is always at the root.
3. **Efficient Operations:**
 - **Insertion:** Inserting a new element into a min-heap takes $O(\log n)$ time.
 - **Extract Minimum:** Removing the smallest element (the root) takes $O(\log n)$ time.
 - **Peek Minimum:** Accessing the smallest element (the root) takes $O(1)$ time.

Consider the following min-heap:

markdown



- **Root Node:** The root node has the smallest value (1).
- **Heap Property:** Every parent node is smaller than its children. For example:
 - Node 1 is smaller than its children 3 and 6.
 - Node 3 is smaller than its children 5 and 9.

Disjoint Set Operations

Set:

A set is a collection of distinct elements. The Set can be represented, for examples, as $S1=\{1,2,5,10\}$.

Disjoint Sets: The disjoint sets are those do not have any common element. For example $S1=\{1,7,8,9\}$ and $S2=\{2,5,10\}$, then we can say that $S1$ and $S2$ are two disjoint sets.

Disjoint Set Operations: The disjoint set operations are

1. UNION &

2. FIND

Disjoint set Union:

If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ consists of all the elements x such that x is in S_i or S_j .

Find:

Example: $S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$ $S_1 \cup S_2 = \{1, 2, 5, 7, 8, 9, 10\}$

Given the element i , find the set containing i .

Example: $S_1 = \{1, 7, 8, 9\}$ Then,

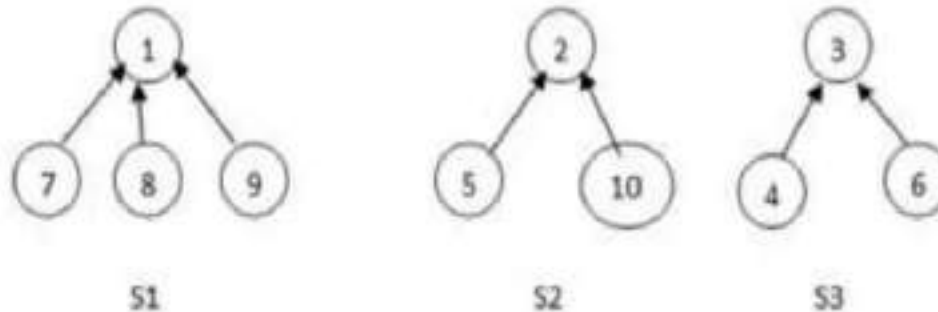
$S_2 = \{2, 5, 10\}$

$S_3 = \{3, 4, 6\}$

Find(4) = S_3 Find(5) = S_2 Find(9) = S_1

Set Representation: The set will be represented as the tree structure where all children will store the address of parent / root node. The root node will store null at the place of parent address. In the given set of elements any element can be selected as the root node, generally we select the first node as the root node.

Example: $S1=\{1,7,8,9\}$ $S2=\{2,5,10\}$ $S3=\{3,4,6\}$ Then these sets can be represented as

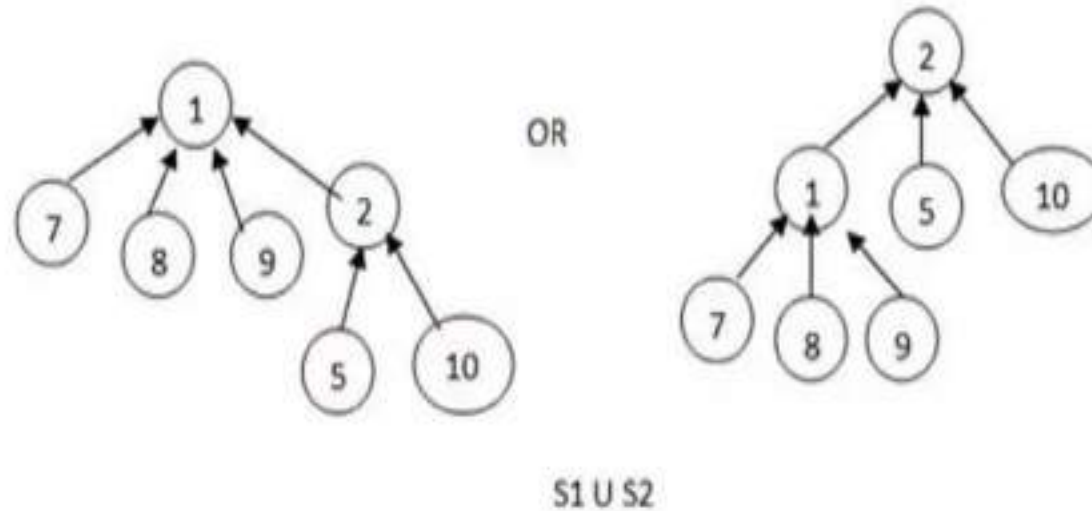


The operations we wish to perform on these sets are:

1. **Disjoint set union.** If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j =$ all elements x such that x is in S_i or S_j . Thus, $S_1 \cup S_2 = \{1, 7, 8, 9, 2, 5, 10\}$. Since we have assumed that all sets are disjoint, we can assume that following the union of S_i and S_j , the sets S_i and S_j do not exist independently; that is, they are replaced by $S_i \cup S_j$ in the collection of sets.
2. **Find(i).** Given the element i , find the set containing i . Thus, 4 is in set S_3 , and 9 is in set S_1 .

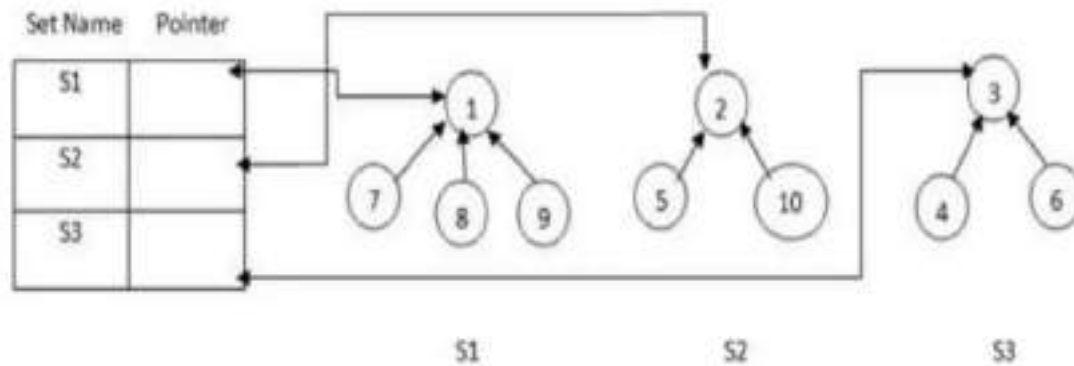
Disjoint Union

Disjoint Union: To perform disjoint set union between two sets S_i and S_j can take any one root and make it sub-tree of the other. Consider the above example sets S_1 and S_2 then the union of S_1 and S_2 can be represented as any one of the following.



Find()

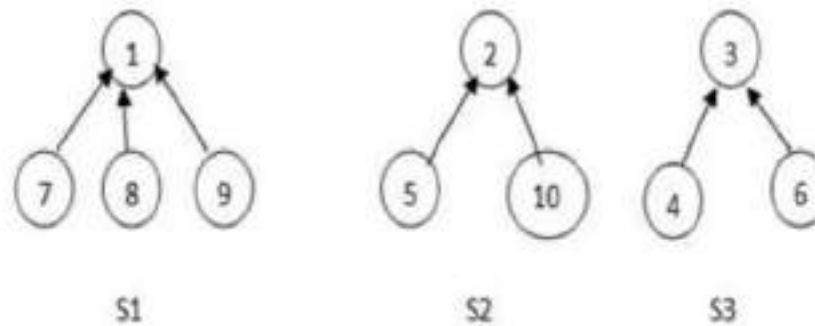
To perform find operation, along with the tree structure we need to maintain the name of each set. So, we require one more data structure to store the set names. The data structure contains two fields. One is the set name and the other one is the pointer to root.



Union and Find Algorithms

Union and Find Algorithms: In presenting Union and Find algorithms, we ignore the set names and identify sets just by the roots of trees representing them. To represent the sets, we use an array of 1 to n elements where n is the maximum value among the elements of all sets. The index values represent the nodes (elements of set) and the entries represent the parent node. For the root value the entry will be '-1'.

Example: For the following sets the array representation is as shown below.



```
1  Algorithm SimpleUnion(i, j)
2  {
3      p[i] := j;
4  }
```

```
1  Algorithm SimpleFind(i)
2  {
3      while (p[i] ≥ 0) do i := p[i];
4      return i;
5  }
```

Algorithm 2.13 Simple algorithms for union and find

Analysis of simple Union, Find

Now let us process the following sequence of *union-find* operations:

$Union(1,2), Union(2,3), Union(3,4), Union(4,5), \dots, Union(n-1,n)$

$Find(1), Find(2), \dots, Find(n)$

This sequence results in the degenerate tree of Figure 2.21.

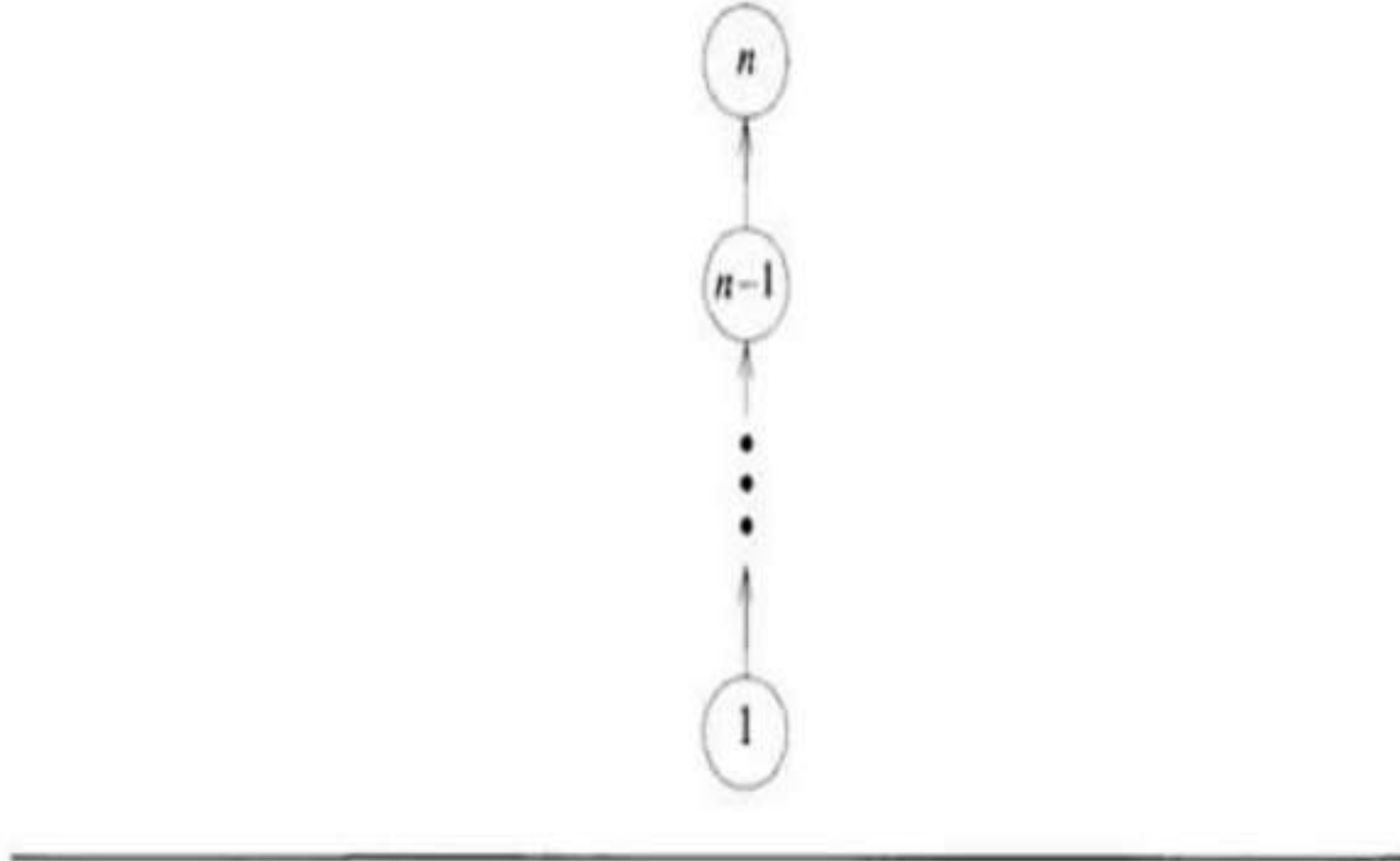


Figure 2.21 Degenerate tree

Weighting rule for Union()

Definition 2.5 [Weighting rule for $Union(i, j)$] If the number of nodes in the tree with root i is less than the number in the tree with root j , then make j the parent of i ; otherwise make i the parent of j . $\square$

Problem

Example 2.4 Consider the behavior of `WeightedUnion` on the following sequence of unions starting from the initial configuration $p[i] = -count[i] = -1, 1 \leq i \leq 8 = n$:

$Union(1, 2), Union(3, 4), Union(5, 6), Union(7, 8),$
 $Union(1, 3), Union(5, 7), Union(1, 5)$

The trees of Figure 2.23 are obtained. As is evident, the height of each tree

Problem

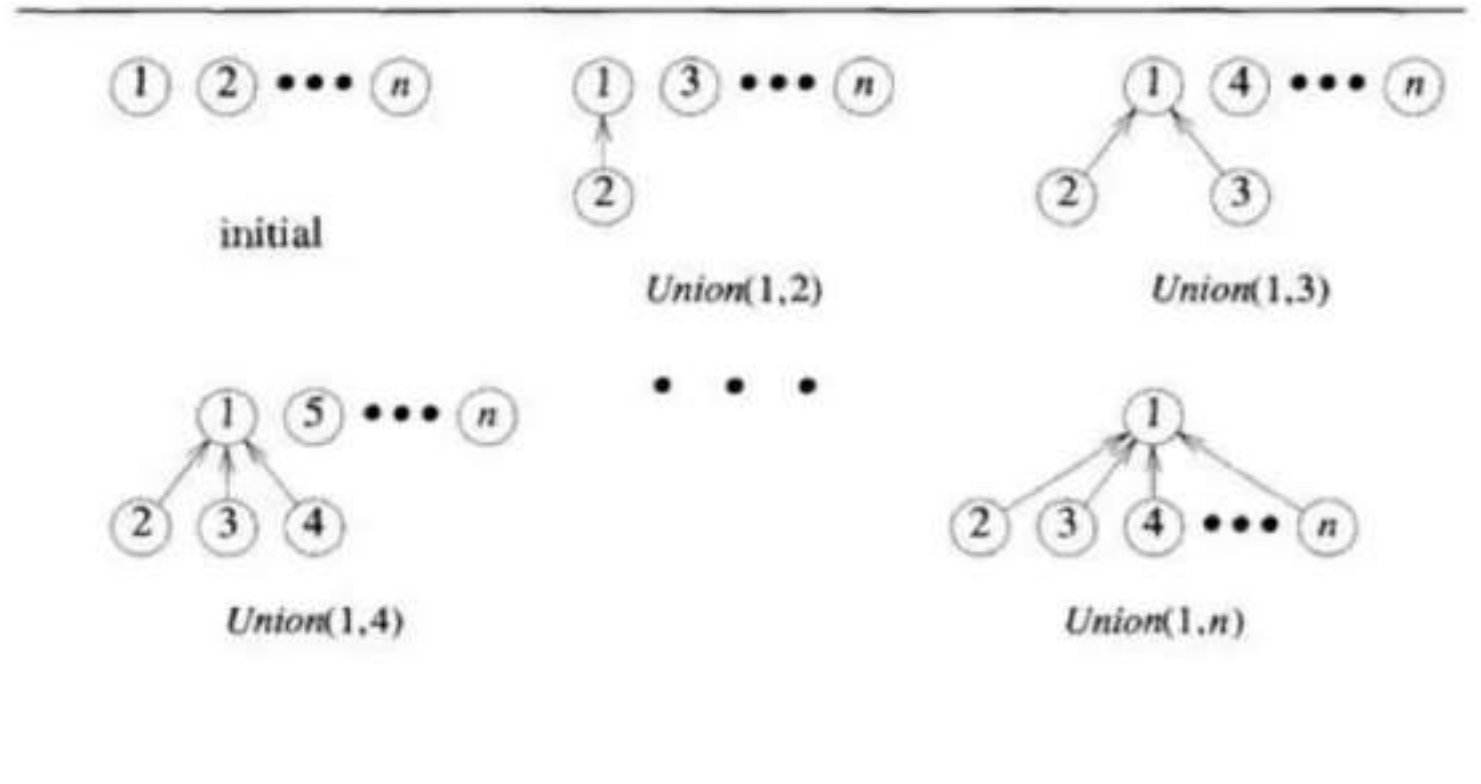


Figure 2.22 Trees obtained using the weighting rule



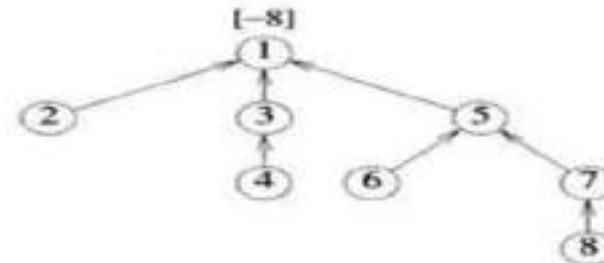
(a) Initial height-1 trees



(b) Height-2 trees following *Union*(1,2), (3,4), (5,6), and (7,8)

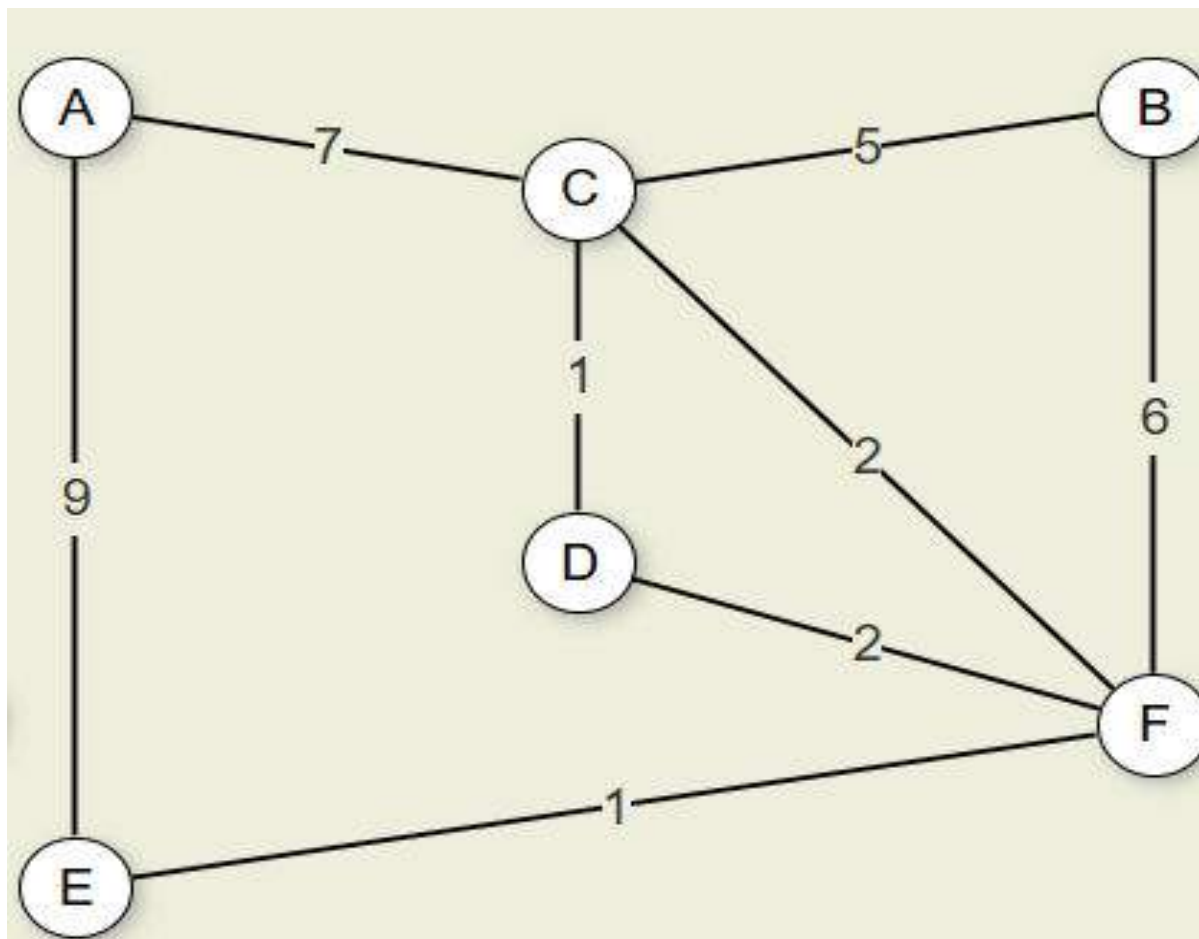


(c) Height-3 trees following *Union*(1,3) and (5,7)



(d) Height-4 tree following *Union*(1,5)

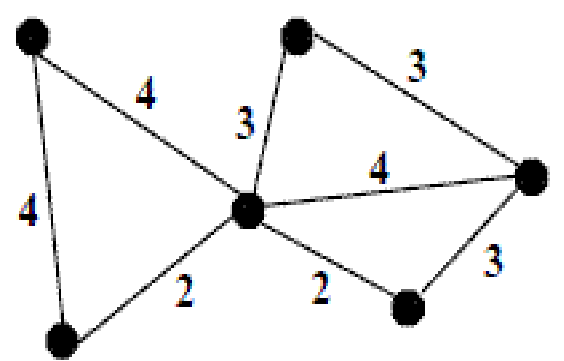
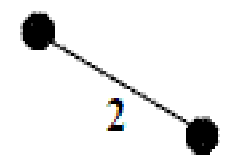
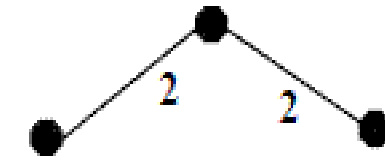
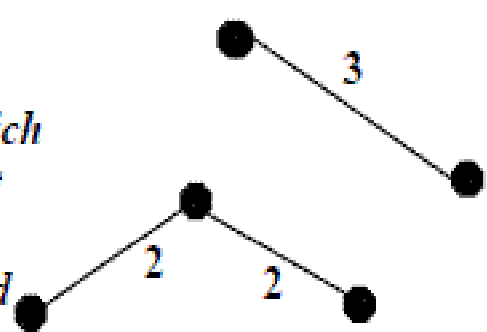
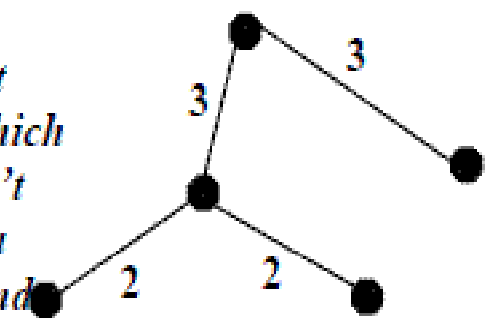
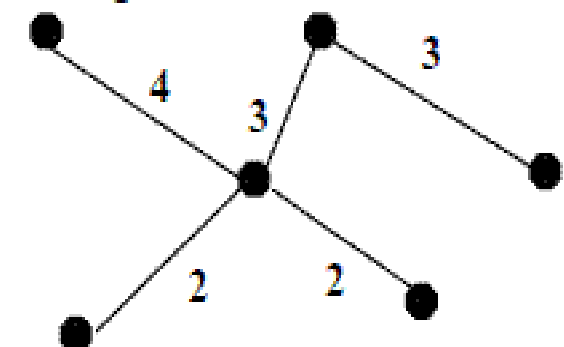
Figure 2.23 Trees achieving worst-case bound



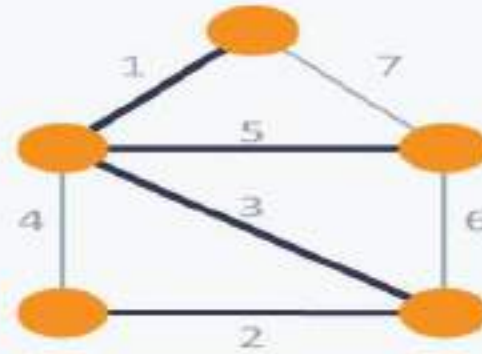
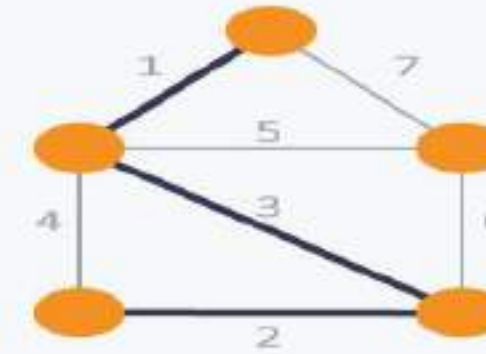
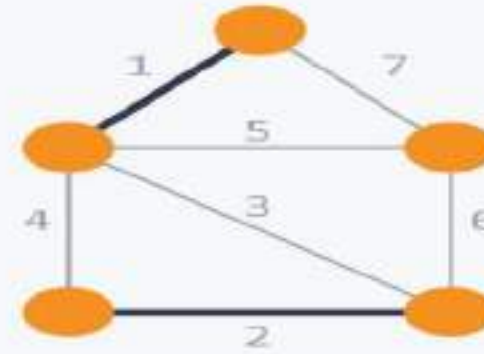
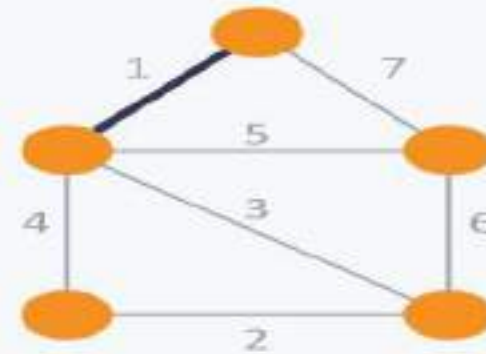
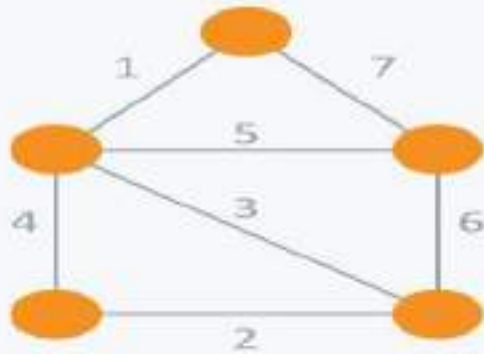
(A,C)	7
(A,E)	9
(C,B)	5
(C,D)	1
(C,F)	2
(D,F)	2
(E,F)	1
(F,B)	6

[https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/Kruskal.html#:~:text=\(A%2CC\)%20\(A%2CE\)%20\(C%2CB\)%20\(C%2CD\)%20\(C%2CF\)%20\(D%2CF\)%20\(E%2CF\)%20\(F%2CB\)](https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/Kruskal.html#:~:text=(A%2CC)%20(A%2CE)%20(C%2CB)%20(C%2CD)%20(C%2CF)%20(D%2CF)%20(E%2CF)%20(F%2CB))

Kruskal's Algorithm

1 <i>Given a network.....</i> 	2 <i>Choose the shortest edge (if there is more than one, choose any of the shortest).....</i> 	3 <i>Choose the next shortest edge and add it.....</i> 
4 <i>Choose the next shortest edge which wouldn't create a cycle and add it.</i> 	5 <i>Choose the next shortest edge which wouldn't create a cycle and add it.</i> 	6 <i>Repeat until you have a minimal spanning tree.</i> 

Kruskal's Algorithm



Early form of minimum cost spanning tree alg

```
t=0;  
while((t has less than n-1 edges) and (E!=0)) do  
{  
  choose an edge (v,w) from E of lowest cost;  
  delete (v,w) from E;  
  if(v,w) does not create a cycle in t then  
    add(v,w) to t;  
  else discard (v,w);  
}
```

MST-Kruskal's Algorithm

Algorithm **kruskal**(E, cost, n, t)

// E is the set of edges in G. G has n vertices. cost[u,v] is the cost of edge(u,v).

// t is the set of edges in the minimum –cost spanning tree. the final cost

// is returned.

//

{

Construct a heap out of the edge costs using Heapify;

for i=1 to n do parent[i]=-1;

//each vertex is in a different set.

i=0;

mincost=0;

```
while((i<n-1) and (heap not empty)) do
{
    delete a minimum cost edge (u,v) from the heap and reheapify using
    Adjust;
    j=Find(u);
    K=Find(v);
    if(j!=k) then
    {
        i=i+1;
        t[I,1]=u;
        t[I,2]=v;
        mincost=mincost+cos[u,v];
        Union(j,k);
    }
}
If(i!=n-1) then
    write("no spanning tree");
else
    return mincost;
```

Kruskal's Algorithm Explanation:

1. Initialization:

- E is the set of edges in the graph G .
- $\text{cost}[u,v]$ is the cost of edge (u,v) .
- n is the number of vertices in the graph.
- t is the set of edges in the minimum-cost spanning tree.
- mincost is used to store the sum of the weights of the edges in the minimum spanning tree.

2. Constructing a Heap:

- The algorithm begins by constructing a heap from the edge costs using a heap construction algorithm (probably **Heapify**). The heap allows efficient extraction of the minimum cost edge.

3. Disjoint Set Initialization:

- An array `parent[]` is initialized to keep track of which set each vertex belongs to. Initially, each vertex is in its own set, so `parent[i] = -1` for all vertices i .

4. Loop for MST Construction:

- The main part of the algorithm is a loop that continues until the number of edges in the spanning tree (i) reaches $n - 1$ (since an MST of n vertices must have exactly $n - 1$ edges), or the heap is empty.

5. Extracting Minimum Cost Edges:

- In each iteration of the loop, the algorithm extracts the edge with the minimum cost from the heap and re-adjusts the heap (`Adjust`).

6. Find and Union Operations (Disjoint Set):

- For each extracted edge (u, v) , the algorithm checks whether the vertices u and v belong to the same set using the `Find` operation. If they do not belong to the same set, the edge is added to the MST (t), and the two sets are merged using the `Union` operation.
- The edge (u, v) is added to the set t , and the cost is added to `mincost`.

7. Termination:

- The loop continues until either $n - 1$ edges have been added to the MST (in which case ``mincost`` is returned), or the heap becomes empty. If the loop ends before adding $n - 1$ edges, it means the graph is not connected, and a spanning tree cannot be formed, so the message "no spanning tree" is printed.

Key Functions:

- **Find(u):** This function finds the root of the set containing vertex u . It is used to check whether two vertices belong to the same set.
- **Union(j,k):** This function merges the sets containing vertices j and k .

Example Walkthrough:

1. Start by initializing all vertices in different sets.
2. Build a heap based on the edge costs.
3. Repeatedly pick the smallest edge and check if adding it would form a cycle (using `Find`). If not, include it in the MST (`Union` the sets).
4. Continue until you have $n - 1$ edges in the MST or there are no more edges in the heap.

Time Complexity:

- Sorting the edges takes $O(E \log E)$ where E is the number of edges.
- The `Find` and `Union` operations are nearly constant time, $O(\log n)$, using efficient data structures like Union-Find with path compression and union by rank.

Kruskal's algorithm is efficient for sparse graphs, especially when the edges are already sorted or can be efficiently sorted.

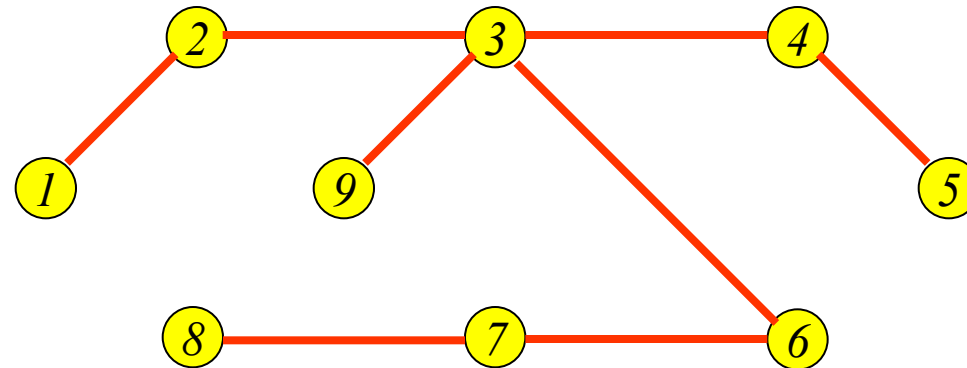
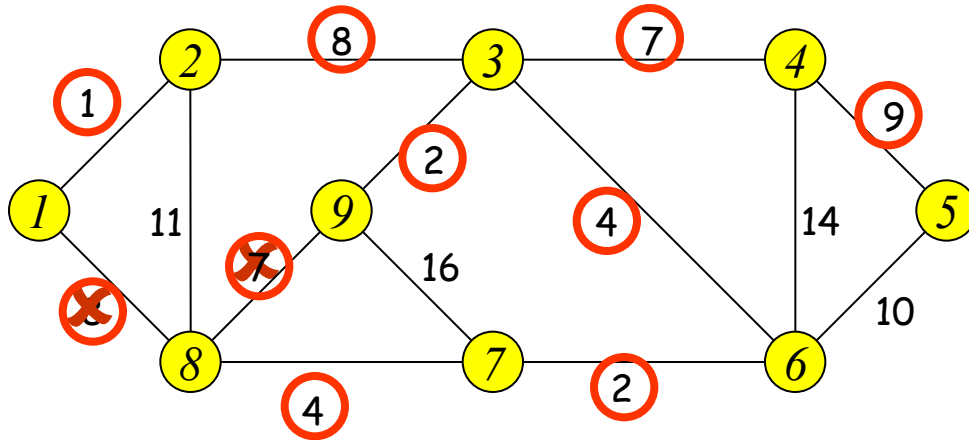
Definition : Let $G(V, E)$ be an undirected connected graph. A subgraph $t = (V, E^I)$ of G is a spanning tree of G iff t is a tree.

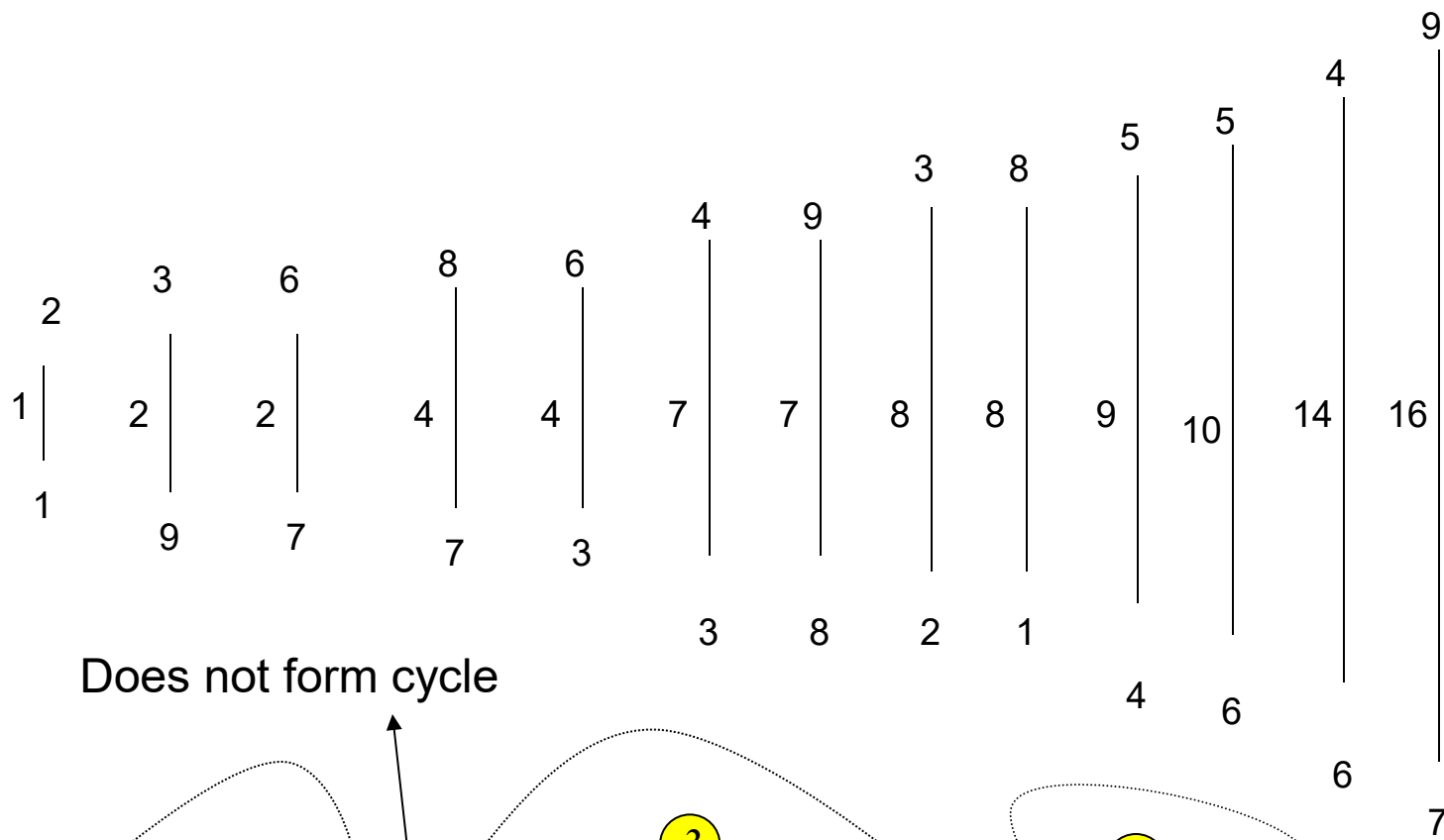
- A *tree* is a connected undirected graph that contains no cycles.
- A *spanning tree* of a graph G is a subgraph of G that is a tree and contains all the vertices of G .

Properties of a Spanning Tree

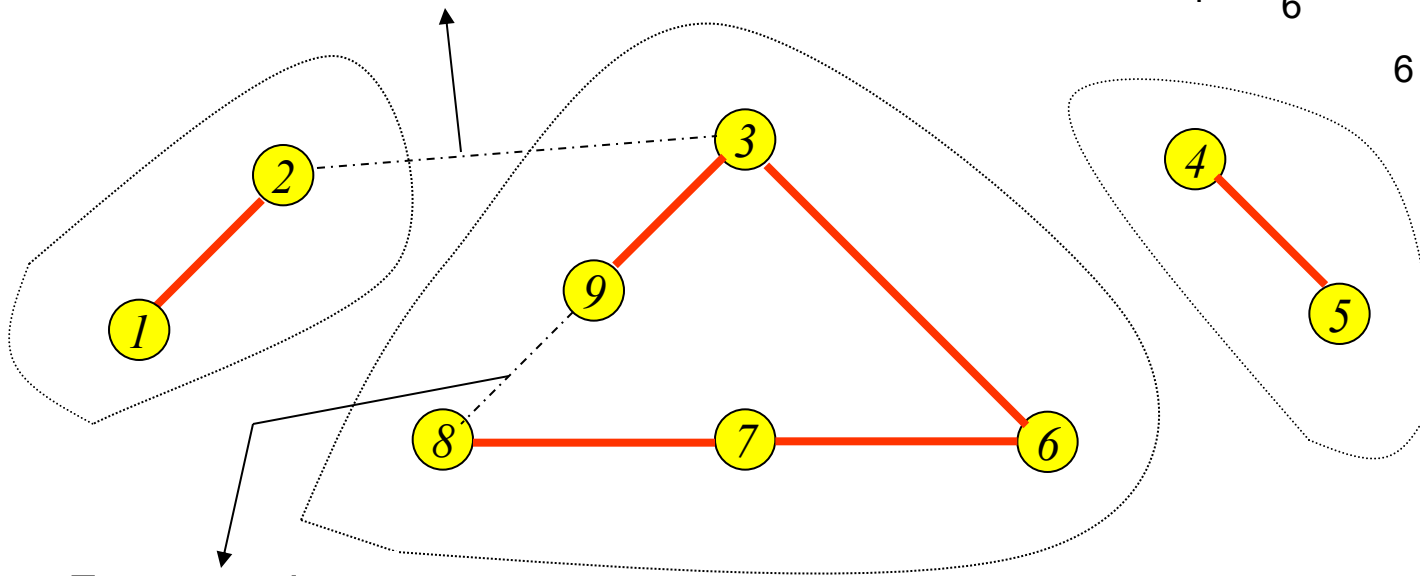
- A connected graph G can have **more than** one spanning tree.
- All possible spanning trees of graph G , have the **same number of vertices**.
- The spanning tree **does not** have any cycle (loops).
- **Removing one** edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- **Adding one edge** to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Kruskal's Algorithm





Does not form cycle



Forms cycle

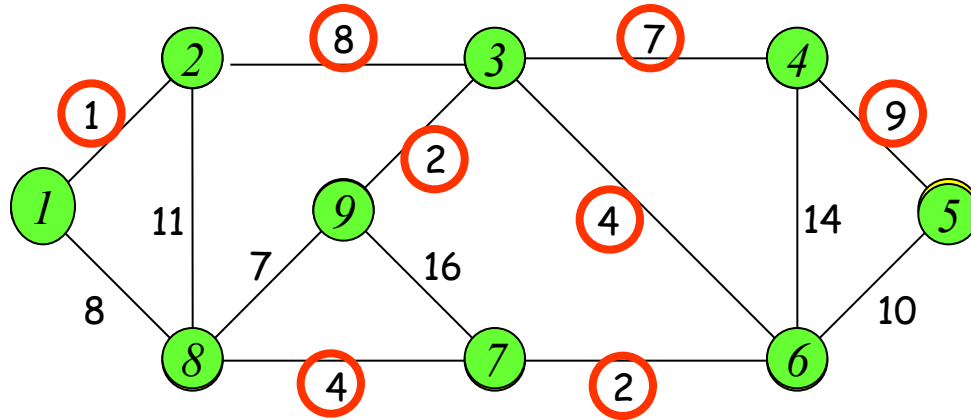
Time complexity of kruskal's algorithm

- With an efficient **Find-set** and **union** algorithms, the running time of kruskal's algorithm will be dominated by the time needed for sorting the edge costs of a given graph.
- Hence, with an efficient sorting algorithm(**merge sort**), the complexity of kruskal's algorithm is $O(E \log E)$.

MST-Prim's Algorithm

- Start with minimum cost edge.
- Select next minimum cost edge (i, j) such that i is a vertex already included in the tree, j is a vertex not yet included.
- Continue this process until the tree has $n - 1$ edges.

Prim's Algorithm



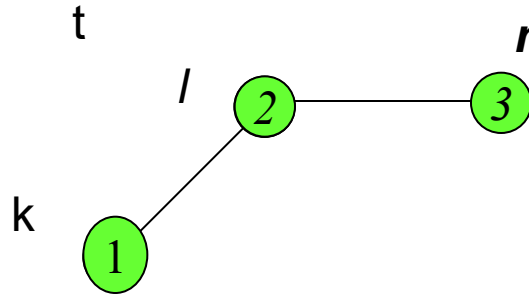
1

2

•
•
•

n-1

1. $near[j]$ is a vertex in the tree such that $cost[j, near[j]]$ is minimum among all choices for $near[j]$



1	$near[1]=0$
2	$near[2]=0$
3	$near[3]=0$

3	$near[3]=2$
4	$near[4]=1$
5	$near[5]=1$
6	$near[6]=1$
7	$near[7]=1$
8	$near[8]=1$
9	$near[9]=1$

$cost[j, near[j]]$

$cost[3, 2]=8$
$cost[4, 1]=\infty$
$cost[5, 1]=\infty$
$cost[6, 1]=\infty$
$cost[7, 1]=\infty$
$cost[8, 1]=8$
$cost[9, 1]=\infty$

2. Select next min cost edge.

Prim's Algorithm

```
1  Algorithm Prim(E, cost, n, t)
2  // E is the set of edges in G.
3  //cost[1:n,1:n] is the cost matrix such that cost[i,j] is either
4  // positive real number or  $\infty$  if no edge (i,j) exists.
   cost[i,j]=0, if i=j.
5  // A minimum spanning tree is computed and stored
6  // as a set of edges in the array t[1:n-1,1:2]
7  {
8      Let (k,l) be an edge of minimum cost in E
9      mincost=cost[k,l];
10     t[1,1]=k; t[1,2]=l;
11     for i=1 to n do // initialize near
12     if( cost[i,1]< cost[i, k] then near[i]=1;
           else near[i]= k;
13     near[k]=near[l]=0;
```

```
14  for i=2 to n-1 do
15      {
16          // Find n-1 additional edges for t.
17          Let j be an index such that near[j]≠0 and
18              cost[j,near[j]] is minimum;
19          t[i,1]=j; t[i,2]=near[j];
20          mincost=mincost+cost[j,near[j]];
21          near[j]=0;
22          for k=1 to n do // update near[]
23              if( ( near[k] ≠0 ) and (cost[k,near[k]>cost[k,j]]) then
24                  near[k]=j;
25      }
26  return mincost;
27 }
```

Input and Output:

- **Input:**
 - E : The set of edges in the graph G .
 - $cost[1:n, 1:n]$: The cost matrix where $cost[i, j]$ is either a positive real number (representing the weight of the edge between vertices i and j), or ∞ if no edge exists between i and j . Additionally, $cost[i, j] = 0$ if $i = j$ (self-loops).
 - n : The number of vertices in the graph.
- **Output:**
 - t : The array of edges representing the minimum spanning tree (MST).
 - $mincost$: The minimum cost of the spanning tree.

Steps of the Algorithm:

1. Initialization:

- The algorithm starts by finding the edge (k, l) with the minimum cost in the entire graph. This edge becomes the first edge of the MST.
- The initial cost of the MST (`mincost`) is set to `cost[k, l]`.
- The edge (k, l) is added to the array `t` which stores the edges of the MST.

2. Initialize the `near[]` Array:

- The `near[]` array is used to store the closest vertex in the MST to each vertex that is not yet included in the MST.
- For every vertex i , the algorithm sets `near[i]` to either k or l , depending on which of $cost[i, k]$ or $cost[i, l]$ is smaller.
- The vertices k and l , which are part of the MST now, are marked as included by setting `near[k] = 0` and `near[l] = 0`.

3. Main Loop to Add $n - 1$ Edges:

- The goal is to add $n - 1$ edges to the MST.
- In each iteration, the algorithm finds the vertex j that is closest to the current MST (i.e., the vertex for which `cost[j, near[j]]` is the minimum among all vertices j such that `near[j]  $\neq$  0`).
- The edge $(j, \text{near}[j])$ is added to the MST, and the cost of this edge is added to `mincost`.
- The vertex j is marked as included in the MST by setting `near[j] = 0`.

4. Update the `near[]` Array:

- After adding a new vertex j to the MST, the algorithm updates the `near[]` array for the remaining vertices.
- For each vertex k , if k is not yet included in the MST (`near[k]  $\neq$  0`) and the cost of the edge (k, j) is smaller than the current closest edge (i.e., `cost[k, near[k]]`), the algorithm updates `near[k]` to be j .

5. Termination:

- The loop continues until $n - 1$ edges have been added to the MST.
- Once the loop is complete, the MST is fully constructed and `mincost` contains the total weight of the MST.
- The algorithm returns `mincost`, which is the minimum cost of spanning the tree.

Prim's Algorithm is another popular greedy algorithm to find the Minimum Spanning Tree (MST) of a connected, undirected graph. Unlike Kruskal's algorithm, which adds edges to the MST based on their weights, Prim's algorithm grows the MST by adding vertices to the tree one by one, always picking the minimum weight edge that connects a vertex inside the MST to a vertex outside it.

Prim's Algorithm Explanation:

1. Initialize Data Structures:

- Start with a graph G having n vertices and E edges.
- You need an array or data structure to track which vertices are already included in the MST.
- A priority queue (or min-heap) is used to always pick the minimum weight edge at each step.

2. Steps:

1. Select a Starting Vertex:

- Begin with any arbitrary vertex u (often the vertex with index 0).
- Add this vertex to the MST.

2. Choose Minimum Weight Edge:

- From the starting vertex u , examine all edges connecting u to its neighboring vertices.
- Select the minimum weight edge that connects u to a vertex v not yet included in the MST.

3. Add Vertex and Edge to the MST:

- Add vertex v and the edge (u, v) to the MST.
- Update the priority queue to include edges from the newly added vertex v to its neighbors.

4. Repeat:

- Continue this process, always selecting the minimum weight edge that connects the growing MST to a new vertex outside the MST.
- This continues until all vertices have been included in the MST.

3. Termination:

- The algorithm ends when $n - 1$ edges have been added to the MST, where n is the number of vertices.
- At this point, the tree includes all vertices and has a total of $n - 1$ edges, which is the definition of a spanning tree.

2. Prim's Algorithm:

- Start from an arbitrary vertex.
- At each step, add the smallest edge that connects a vertex inside the growing MST to a vertex outside it.
- Repeat until all vertices are included.

Prim(G , cost, n)

// G is the graph, cost[u , v] is the cost of edge (u , v)

// n is the number of vertices

// Returns the minimum cost of the spanning tree

Let $T = \emptyset$ // T will store the edges of the MST

Initialize a priority queue (min-heap) Q

Let selected[] = False for all vertices // To track vertices in MST

Start from any vertex, say vertex 0

Add all edges from vertex 0 to the priority queue Q

While T does not contain $n - 1$ edges do

 Select the edge (u, v) with the smallest cost from Q

 If vertex v is not already in the MST (i.e., `selected[v] == False`)
then

 Add vertex v to the MST

 Add edge (u, v) to T

 Mark `selected[v] = True`

 Add all edges from vertex v to the priority queue Q

 Remove edge (u, v) from the queue

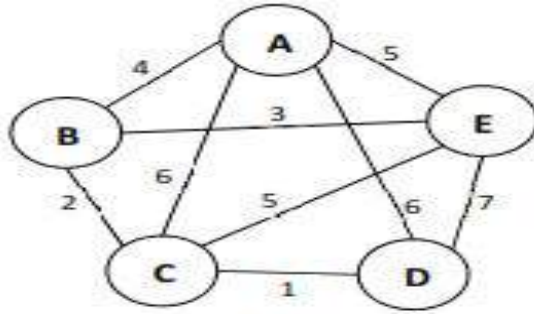
Return the edges in T and the minimum cost

Time Complexity:

- Using a binary heap, Prim's algorithm runs in $O((E + V) \log V)$, where E is the number of edges and V is the number of vertices.
- The complexity can be reduced further to $O(E \log V)$ using Fibonacci heaps.

Differences from Kruskal's Algorithm:

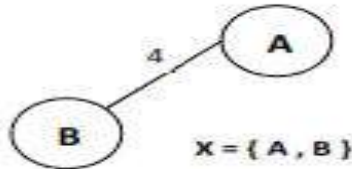
- **Approach:** Prim's algorithm grows the MST one vertex at a time, always choosing the smallest edge connected to the current MST, while Kruskal's algorithm grows the MST by adding the smallest edge that does not form a cycle.
- **Efficiency:** Prim's algorithm is more efficient than Kruskal's algorithm when dealing with dense graphs (many edges), while Kruskal's is better for sparse graphs.



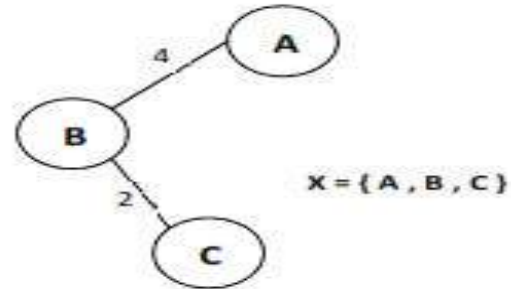
Using Prim's Algorithm:

Let X be the set of nodes explored, initially $X = \{A\}$

Step 1: Taking minimum weight edge of all Adjacent edges of $X = \{A\}$

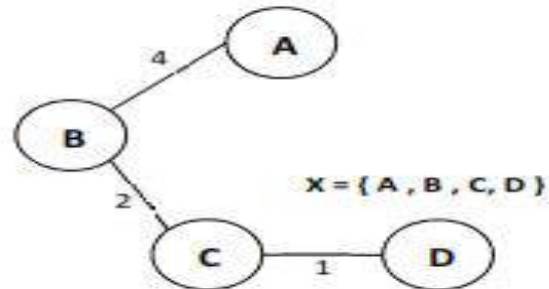


Step 2: Taking minimum weight edge of all Adjacent edges of $X = \{A, B\}$

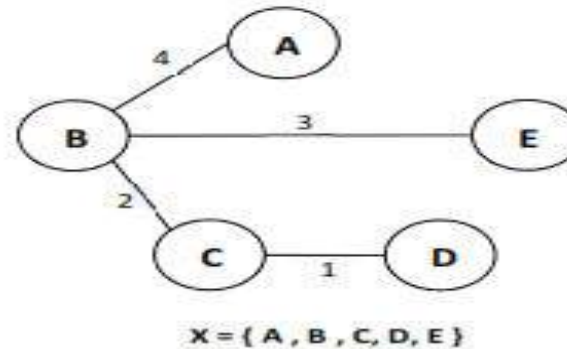


A - B	4
A - E	5
A - C	6
A - D	6
B - E	3
B - C	2
C - E	6
C - D	1
D - E	7

Step 3: Taking minimum weight edge of all Adjacent edges of $X = \{A, B, C\}$

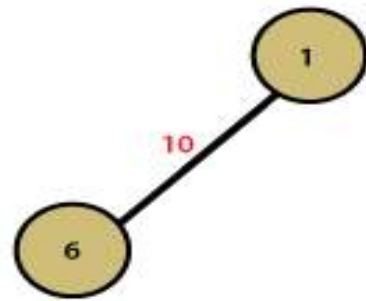
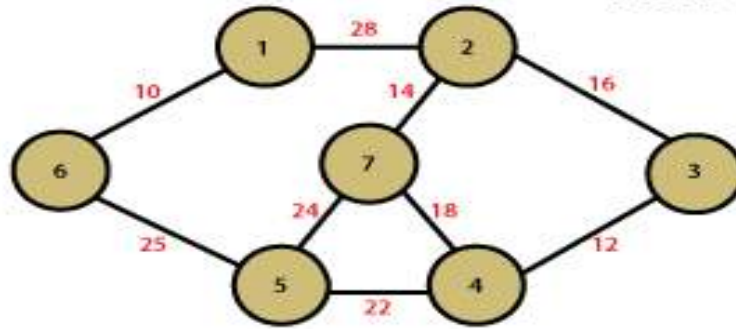


Step 4: Taking minimum weight edge of all Adjacent edges of $X = \{A, B, C, D\}$

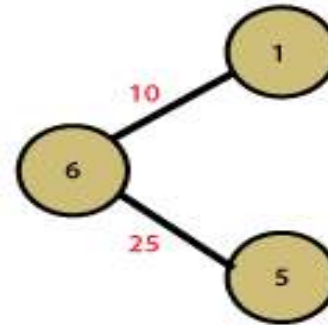


All nodes of graph are there with set X, so we obtained minimum spanning tree of cost: $4 + 2 + 1 + 3 = 10$

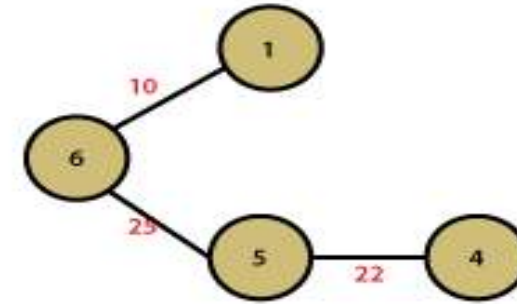
Prim's Algorithm



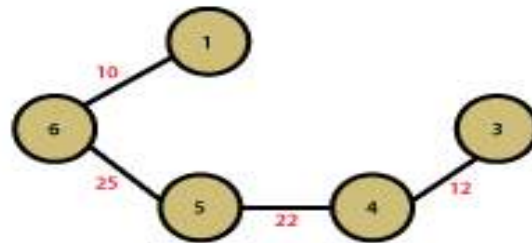
Step 1



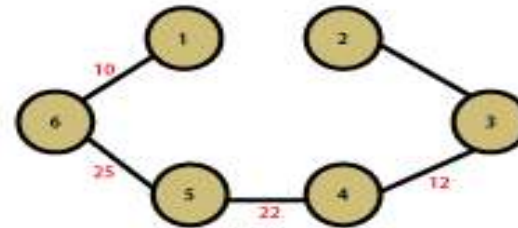
Step 2



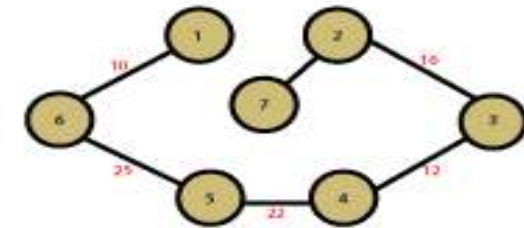
Step 3



Step 4

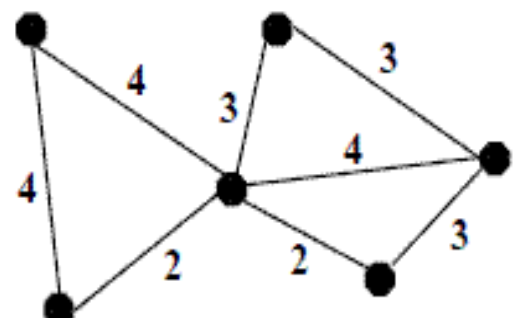
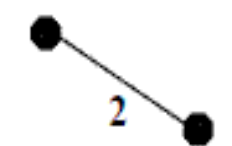
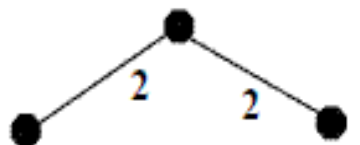
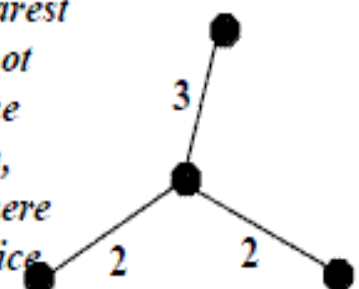
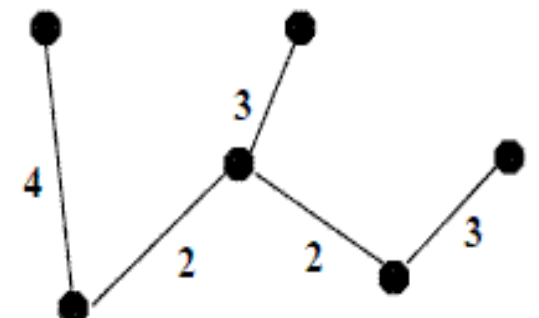


Step 5



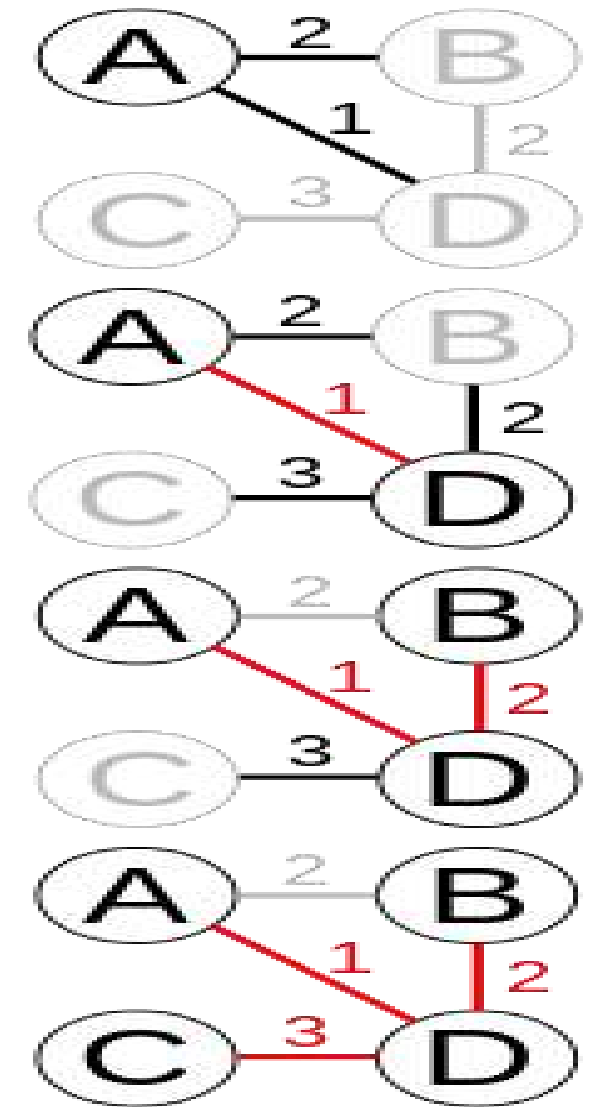
Step 6

Prim's Algorithm

1 <i>Given a network.....</i> 	2 <i>Choose a vertex</i> 	3 <i>Choose the shortest edge from this vertex.</i> 
4 <i>Choose the nearest vertex not yet in the solution.</i> 	5 <i>Choose the next nearest vertex not yet in the solution, when there is a choice choose either.</i> 	6 <i>Repeat until you have a minimal spanning tree.</i> 

Prim's algorithm (also known as Jarník's algorithm) is a greedy algorithm that finds a minimum spanning tree for a weighted undirected graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized.

- The algorithm operates by building this tree one vertex at a time, from an arbitrary starting vertex, at each step adding the cheapest possible connection from the tree to another vertex.
- Following Prim's algorithm starting at vertex A. Then among all the near by edges, the smallest one for A is AD. In the third step, edges BD and AB both have weight 2, so BD is chosen arbitrarily. After that step, AB is no longer a candidate for addition to the tree because it links two nodes that are already in the tree.



Differences



Kruskal's Algorithms	Prim's Algorithm
1. Select the shortest edge in a network 2. Select the next shortest edge which does not create a cycle 3. Repeat step 2 until all vertices have been connected 4. Kruskal's Begins with forest and merge into tree.	1. Select any vertex 2. Select the shortest edge connected to that vertex 3. Select the shortest edge connected to any vertex already connected 4. Repeat step 3 until all vertices have been connected 5. Prim's always stays as a tree.
<u>Complexity</u> $O(N \log N)$ comparison sort for edges.	<u>Complexity</u> $O(N \log N)$ search the least weight edge for every vertices.

Future Work and Scope

Kruskal's Algorithm

Even after connection of all vertices into tree , alg will be continued till meeting $n-1$ edges .

Prim's Algorithm

Alg need to calculate cost of remaining edges even though its makes cycle/connected.

Sample questions

- Define a tree and spanning tree
- Constraints used in prims algorithm
- Constraints used in kruskal's algorithm
- Analyse the time complexities of prims and kruskals

OR

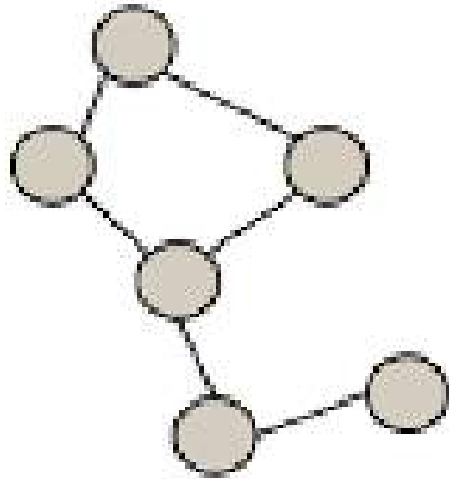
Spanning Tree

- **Graph:** Set of Vertices and edges $G: \{ V, E \}$
- A *tree* is a connected undirected graph that contains **no cycles**.
- i.e **Tree** : Tree is Graph $T: \{ V, E \}$
 - Acyclic
 - N nodes
 - N-1 edges
- Note: Every Tree is a Graph , But Every Graph is not a Tree

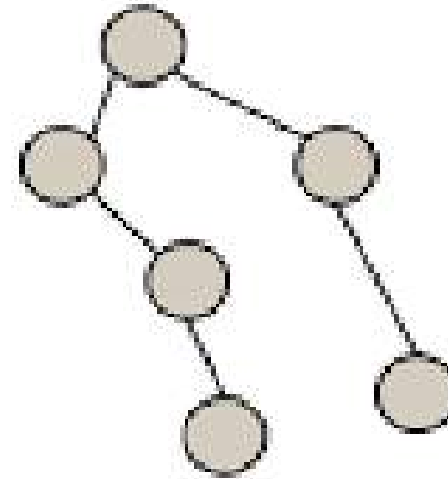
Properties of a Spanning Tree

- A connected graph G can have **more than** one spanning tree.
- All possible spanning trees of graph G , have the **same number of edges and vertices**.
- The spanning tree **does not** have any cycle (loops).
- **Removing one** edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- **Adding one edge** to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

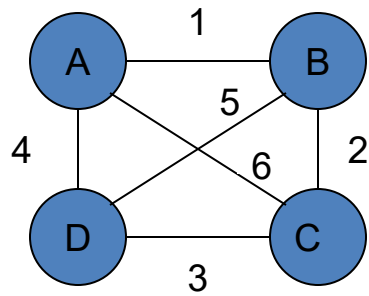
Graphs & Trees



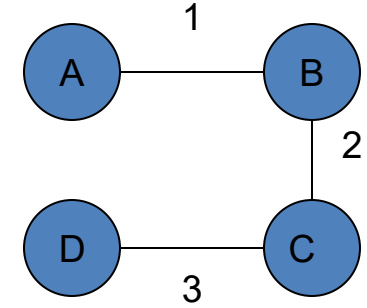
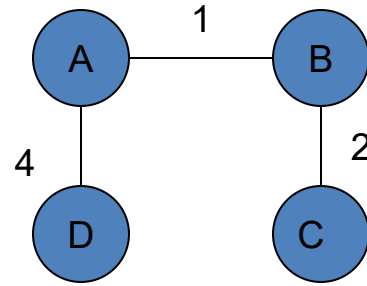
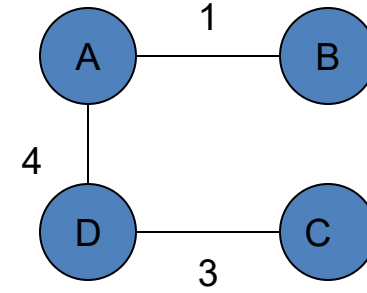
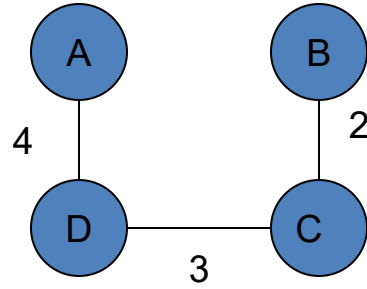
Graph



Tree



Undirected Graph



...

Some Spanning Trees

Spanning Tree : A spanning tree of G is a sub graph T that is

➤ Connected.

➤ Acyclic.

➤ Includes all of the vertices in given graph and vertices-1 edges.

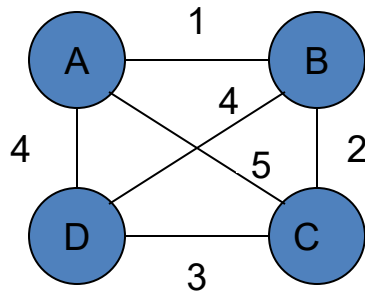
- $T \subseteq G$ $T: \{V' = V, E' \subseteq E\}$
 $V' = V$ $E = |V| - 1$

Spanning Tree properties:

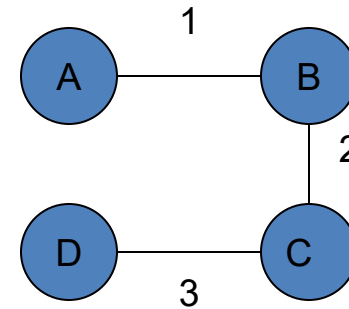
- A graph may have many spanning trees. If the Graph $G: \{V, E\}$ then the possible spanning Trees are $ECV-1$
- Removing one edge from the Spanning Tree Will make it as Disconnected graph
- Adding one edge to the Spanning Tree will create a cycle.
- If each edge has distinct weight then there will be only one & unique Minimum cost Spanning Tree.
- Disconnected graph doesn't have any Spanning Tree.

Minimum Cost Spanning Tree (MST)

- A **minimum spanning tree** is the one among all the spanning trees with the **smallest total cost** or A Spanning tree with minimum weight



Undirected Graph

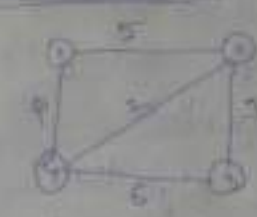


Minimum Spanning Tree

Applications of MSTs

- Computer Networks
 - How to connect a set of computers using the **minimum amount of wire..**
- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis

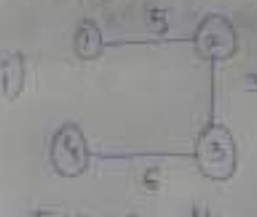
Weighted graphs:





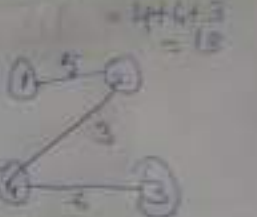
mc = Minimum cost
 $= 4 + 4 + 3 = 11$

(ii)



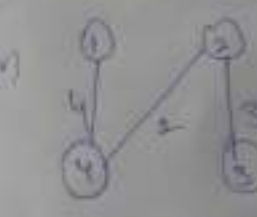
$5 + 3 + 6 = 14$

(iii)



$5 + 3 + 6 = 14$

(iv)



$4 + 2 + 3 = 9$

From above observation, if you have a weighted graph and get different spanning tree with varying in minimum cost
 $\Rightarrow$ For all different spanning tree which is minimum that is minimum cost spanning tree

→ Trying all possible spanning trees
with minimum cost is too lengthy.
That's why go for Greedy Method
with Two Algorithms

(i) Prim's Algorithm

(ii) Kruskal's Algorithm

Kruskal's Algorithm

- Kruskal's algorithm is a greedy algorithm in graph theory that finds a minimum spanning tree for a connected weighted graph.
- This algorithm treats every node as an independent tree and connects one with another only if it has the lowest cost compared to all other options available.
- **Steps to Kruskal's algorithm:**
 - Sort the graph edges with respect to their weights.
 - Start adding edges to the minimum spanning tree from the edge with the smallest weight until the edge of the largest weight.
 - Only add edges which don't form a cycle—edges which connect only disconnected components.

Kruskal's Algorithm:

MST KRUSKAL(G, w)

$T \leftarrow \emptyset$

$\text{cost} \leftarrow 0$

for each vertex $v \in V[G]$

do MAKE-SET(v)

sort the edges of E into nondecreasing order by weight w

for each edge $(u, v) \in E$, taken in nondecreasing order by weight

do if FIND-SET(u) $\neq$ FIND-SET(v)

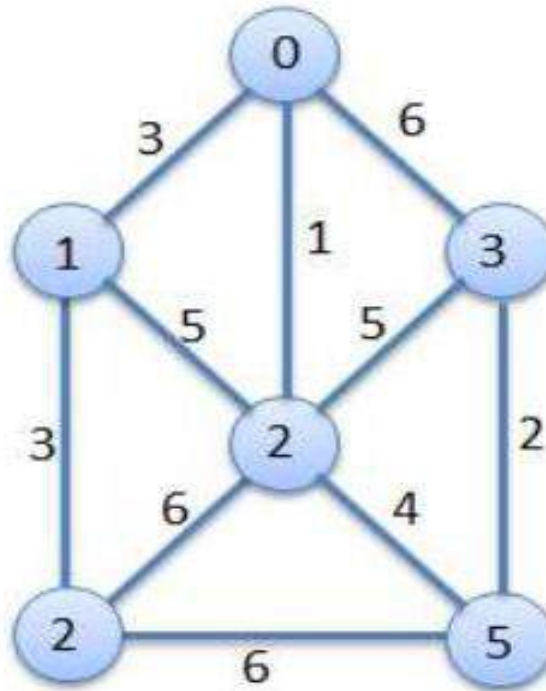
then $T.E \leftarrow T.E \cup \{(u, v)\}$

UNION(u, v)

$\text{cost} \leftarrow \text{cost} + \text{cost}(u, v)$

return cost

Example: Find Minimum spanning tree cost using kruskal's method for the following Graph

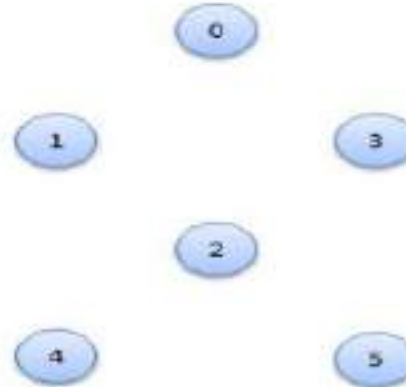


Step1. Edges are sorted in ascending order by weight.

Edge No.	Vertex Pair	Edge Weight
E1	(0,2)	1
E2	(3,5)	2
E3	(0,1)	3
E4	(1,4)	3
E5	(2,5)	4
E6	(1,2)	5
E7	(2,3)	5
E8	(0,3)	6
E9	(2,4)	6
E10	(4,5)	6

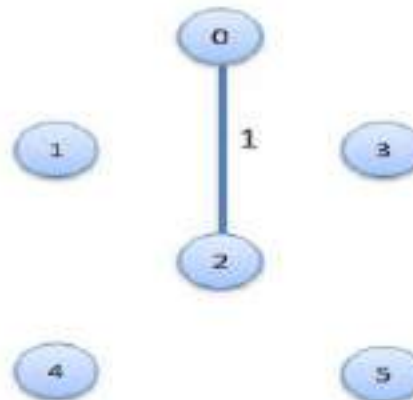
Step2. Edges are added in sequence.

$T.E = \{ \emptyset \}$ cost=0

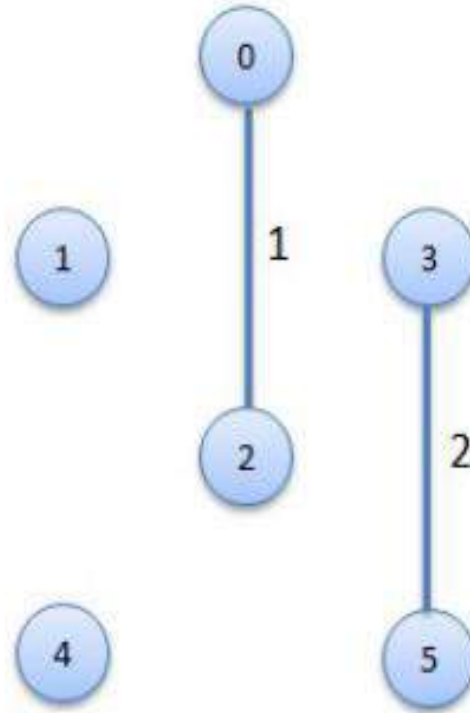


Step3: Consider the minimum edge if it's not connected add the edge and consider the cost

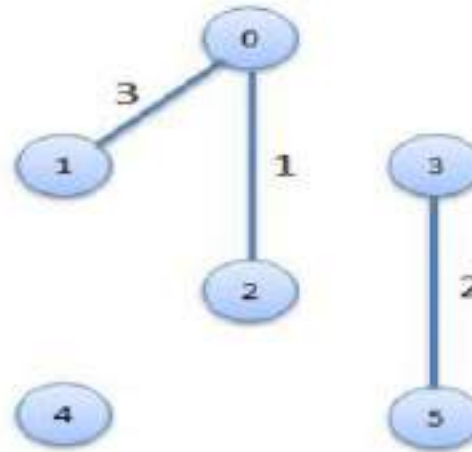
Edge E1 $T.E = \{ (0,2) \}$ cost=1



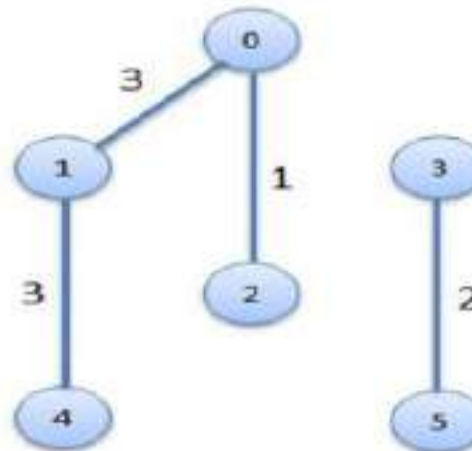
Step4: consider the Edge E2 T.E= { (0,2),(3,5) } cost=1+2



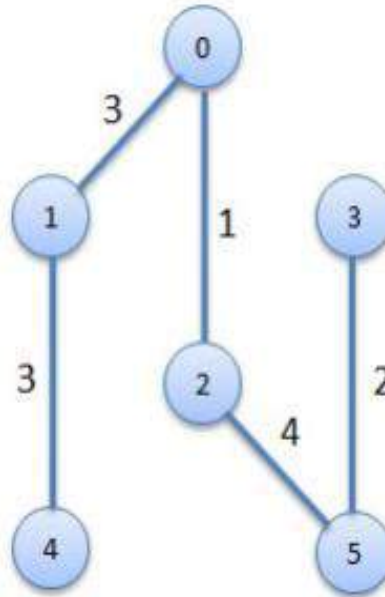
Step5: consider the Edge E3 T.E= { (0,2), (3,5), (1,0) } cost=1+2+3



Step6: consider the Edge E4 T.E= { (0,2), (3,5), (1,0), (1,4) } cost=1+2+3+3



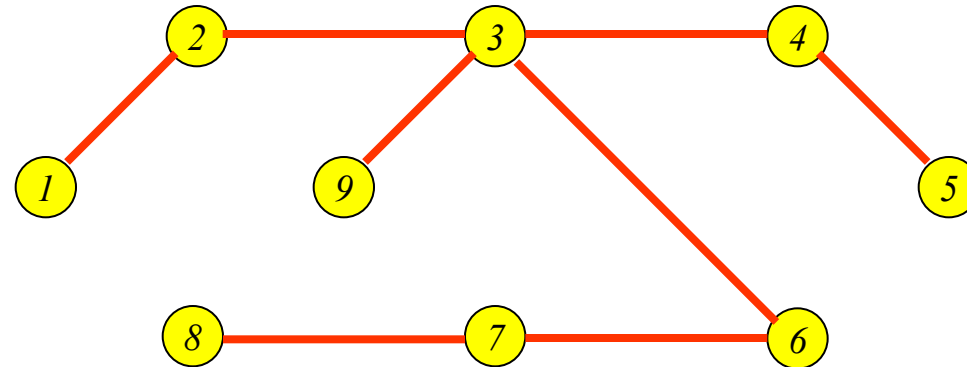
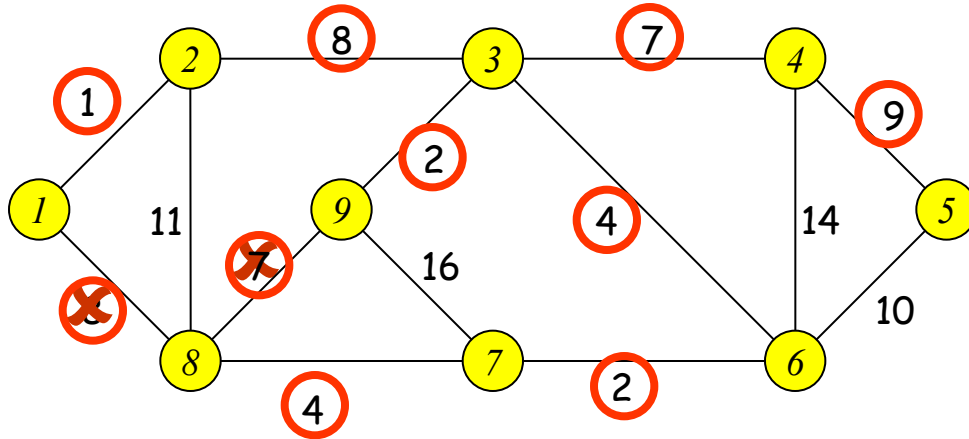
Step7: consider the Edge E5 T.E= { (0,2), (3,5), (1,0), (1,4), (2,5) } cost=1+2+3+3+4

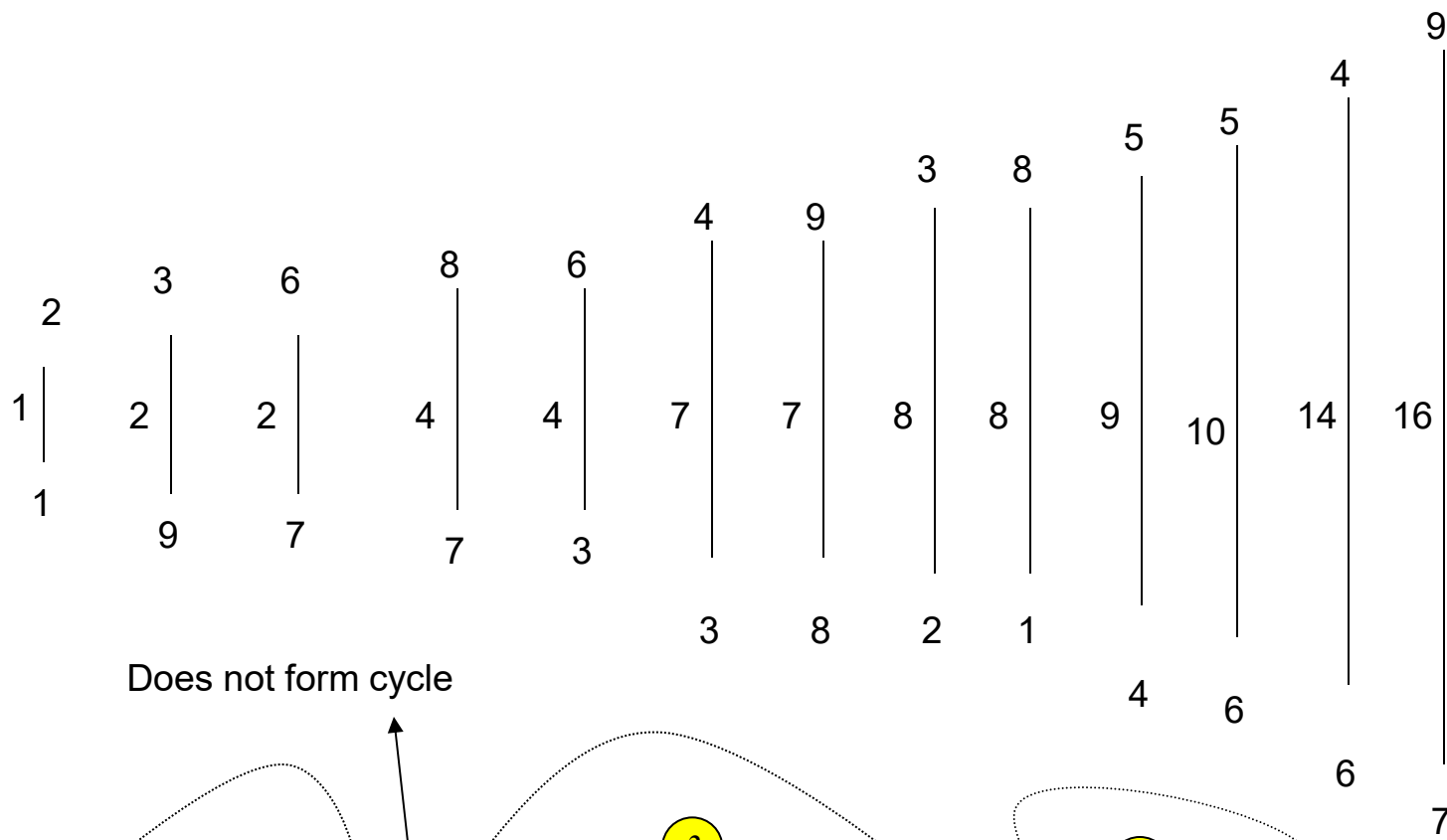


Step 8: check all the remaining edges such that it doesn't form any cycle, for all remaining if we check it will form cycle and its already connected component so the cost of the Tree is:

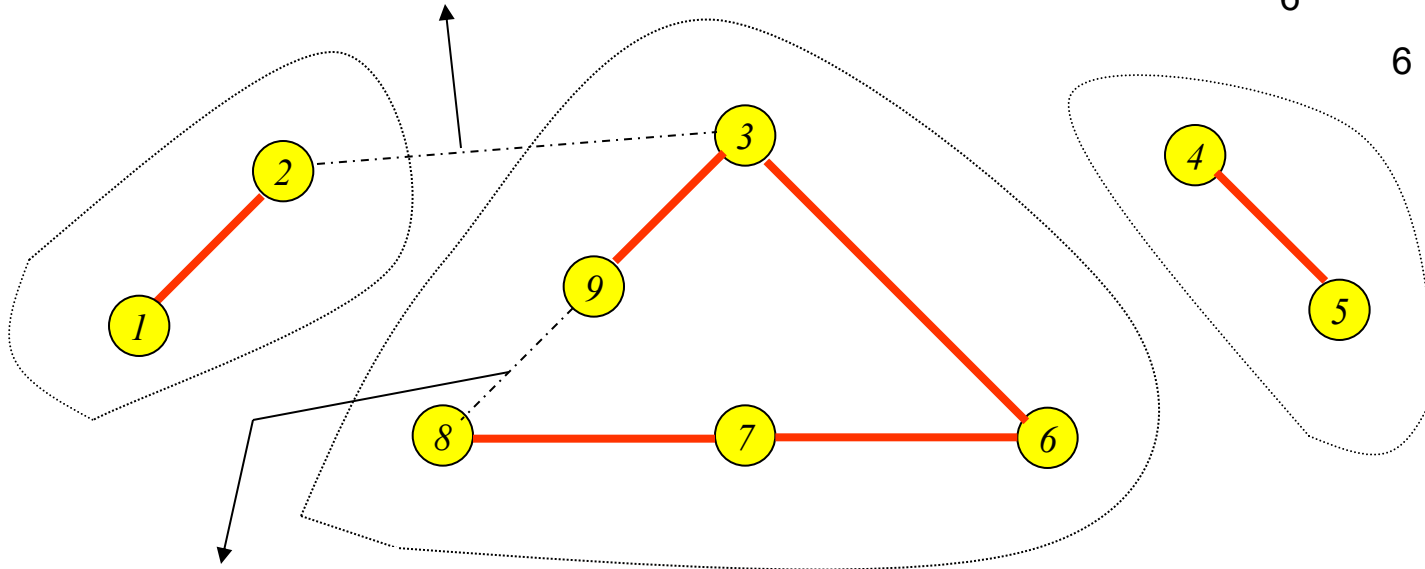
$$\text{Total Cost} = 1+2+3+3+4 = 13$$

Kruskal's Algorithm





Does not form cycle



Forms cycle

Time complexity of kruskal's algorithm

- With an efficient **Find-set** and **union** algorithms, the running time of kruskal's algorithm will be dominated by the time needed for sorting the edge costs of a given graph.
- Hence, with an efficient sorting algorithm(**merge sort**), the complexity of kruskal's algorithm is $O(E \log E)$.

MST-Prim's Algorithm

- Prim's Algorithm is another greedy algorithm used for finding the Minimum Spanning Tree (MST) of a given graph.
- The graph must be weighted, connected and undirected
- Start with minimum cost edge.
- For rest of the procedure, always select a minimum cost edge from graph make sure that already connected to the selected vertices.
- Continue this process until the tree has $n - 1$ edges.

Algorithm for prim's

```
root ← {Any arbitrary node in the G.V}
Prim(G={ V,E},root) {
  T ← ∅
  T.V ← T.V ∪ root
  Cost ← 0
  While (G.V ≠ T.V) do
  {   find an edge e: u→v  of min cost
      such that u∈ T.V and v ∉ T.V
      T ← T.E ∪ (e)
      T.V ← T.V ∪ (v)
      cost ← cost +cost(u,v)
  }
  return cost
}
```

Steps for implementing Prim's Algorithm-

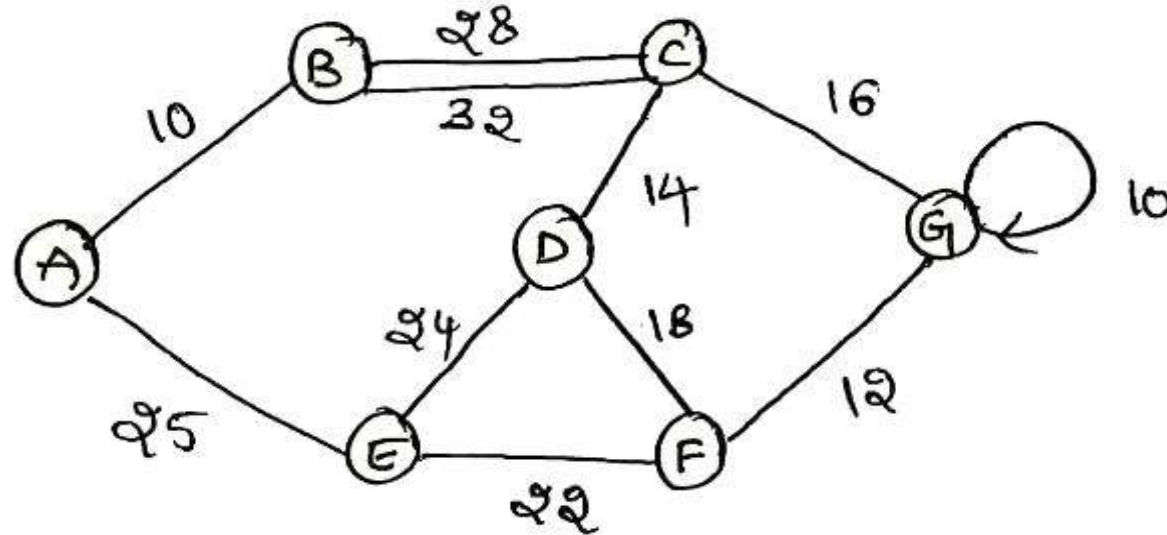
Step-01: Remove all loops and parallel edges in graph if there are any in given graph then apply prim's algorithm

Step-02: Randomly choose any vertex. We usually select and start with a vertex that connects to the edge having least weight.

Step-03: Find all the edges that connect the tree to new vertices, then find the least weight edge among those edges and include it in the existing tree. If including that edge creates a cycle, then reject that edge and look for the next least weight edge.

Step-04: Keep repeating step-03 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

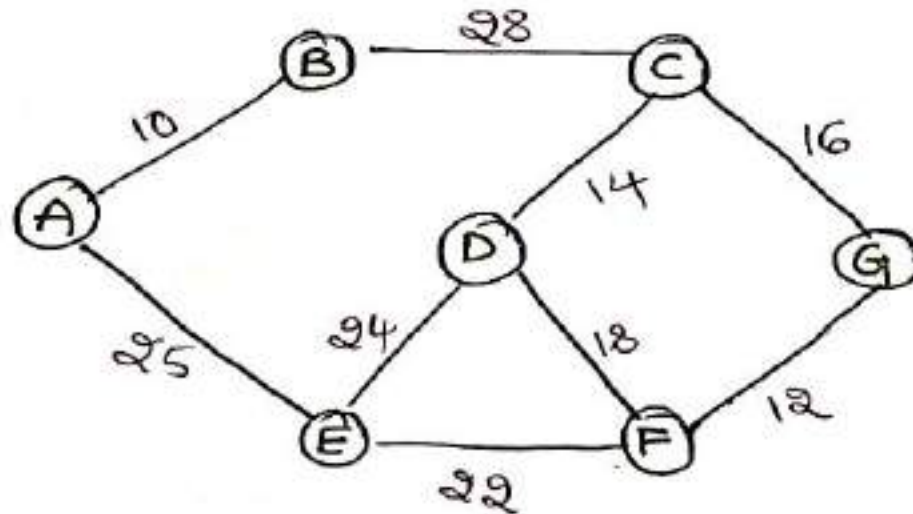
Question: Construct the minimum spanning tree
(CMST) for the given graph using prims algorithm?



Solution:

Solution:

Step 1: Remove self loops and parallel edge, if any present in the graph. If any parallel edge is there consider minimum cost edge, redraw the graph.



step 2 : Select arbitrary node as root Here

step 2 : Select arbitrary node as root Here

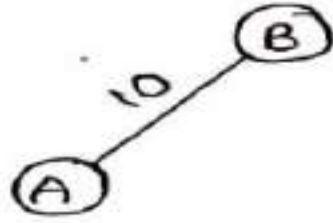
$$\text{root} \leftarrow A$$

$$T.V = \{A\} \quad \text{i.e. visited node}$$

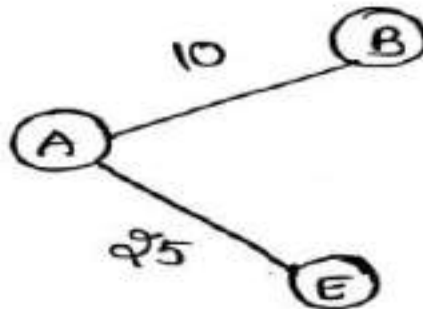
$$G.V = \{A, B, C, D, E, F, G\}$$

$$T.E = \{(A|B)\}$$

$$\text{Cost} = 10$$



STEP 3: $T.V = \{A, B, E\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T.E = \{(A, B), (A, E)\}$
 $COST = 10 + 25$



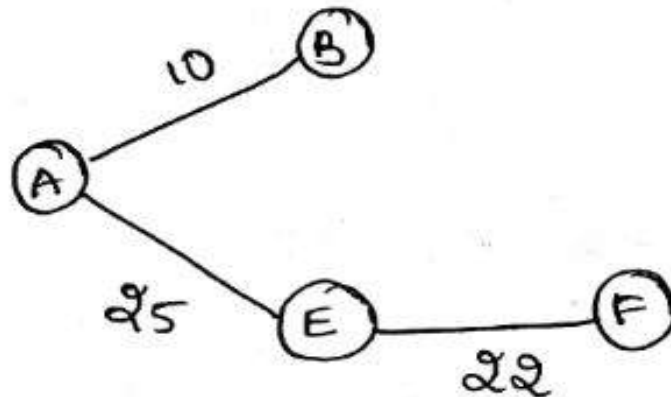
step 4:

$$T.V = \{ A, B, E, F \}$$

$$G.V = \{ A, B, C, D, E, F, G \}$$

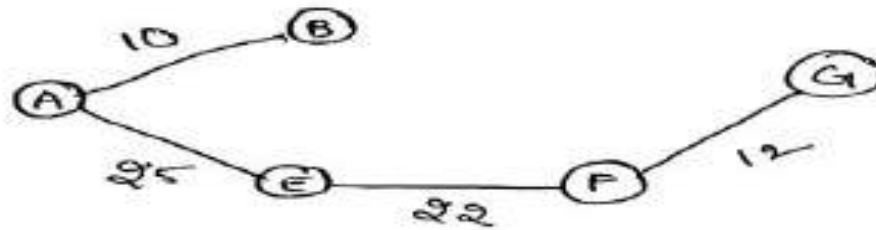
$$T.E = \{ (A, B), (A, E), (E, F) \}$$

$$\text{Cost} = 10 + 25 + 22$$



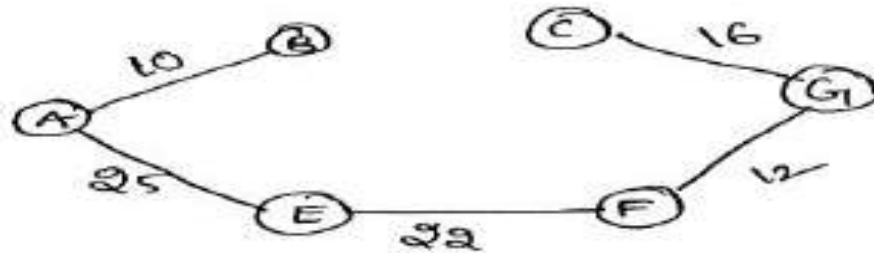
Step 5: $T.V = \{A, B, E, F, G\}$
 $G.V = \{A, B, C, D, E, F, G\}$

Step 5: $T.V = \{A, B, E, F, G\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T-E = \{(A, B), (A, E), (E, F), (F, G)\}$
 $Cost = 10 + 25 + 22 + 12$



Step 6:

$T.V = \{A, B, E, F, G, C\}$
 $G.V = \{A, B, C, D, E, F, G\}$
 $T-E = \{(A, B), (A, E), (E, F), (F, G), (G, C)\}$
 $Cost = 10 + 25 + 22 + 12 + 16$



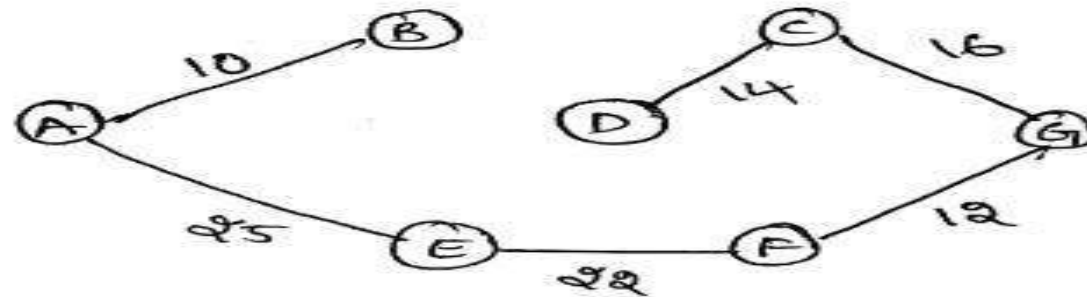
Step 7:

$$T.V = \{ A, B, E, F, G, C \}$$

$$G.V = \{ A, B, C, D, E, F, G \}$$

$$T.E = \{ (A, B), (A, E), (E, F), (F, G), (G, C), (C, D) \}$$

$$\text{Cost} = 10 + 25 + 22 + 12 + 16 + 14$$



Step 8:

All vertices covered the final cost

$$is = 10 + 25 + 22 + 12 + 16 + 14$$

$$= 99 \text{ units.}$$

Question-01:

Will both Prim's Algorithm and Kruskal's Algorithm always produce the same Minimum Spanning Tree (MST) for any given graph?

Question-01:

Will both Prim's Algorithm and Kruskal's Algorithm always produce the same Minimum Spanning Tree (MST) for any given graph?

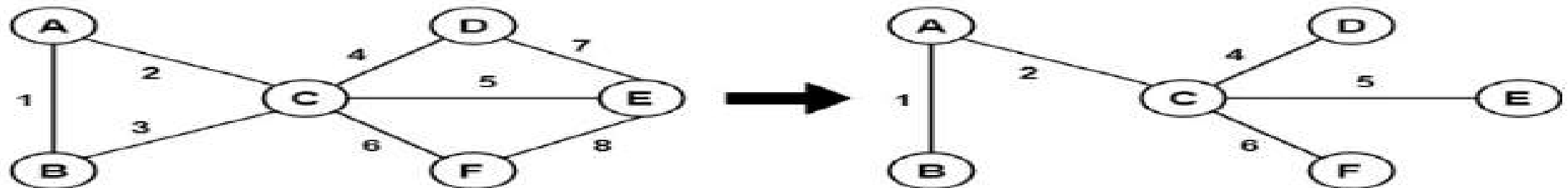
The following two cases are possible-

Case-01: When all the edge weights are distinct-

If all the edge weights are distinct, then both the algorithms are guaranteed to find the same i.e. unique MST.

Example-

Consider the following example-



Minimum Spanning Tree (MST)
(Cost = 18 units)

The application of both the algorithms on the above graph will produce the same MST as shown.

This example clearly illustrates when all the edge weights are distinct, both the algorithms always produces the same MST having the same cost as shown.

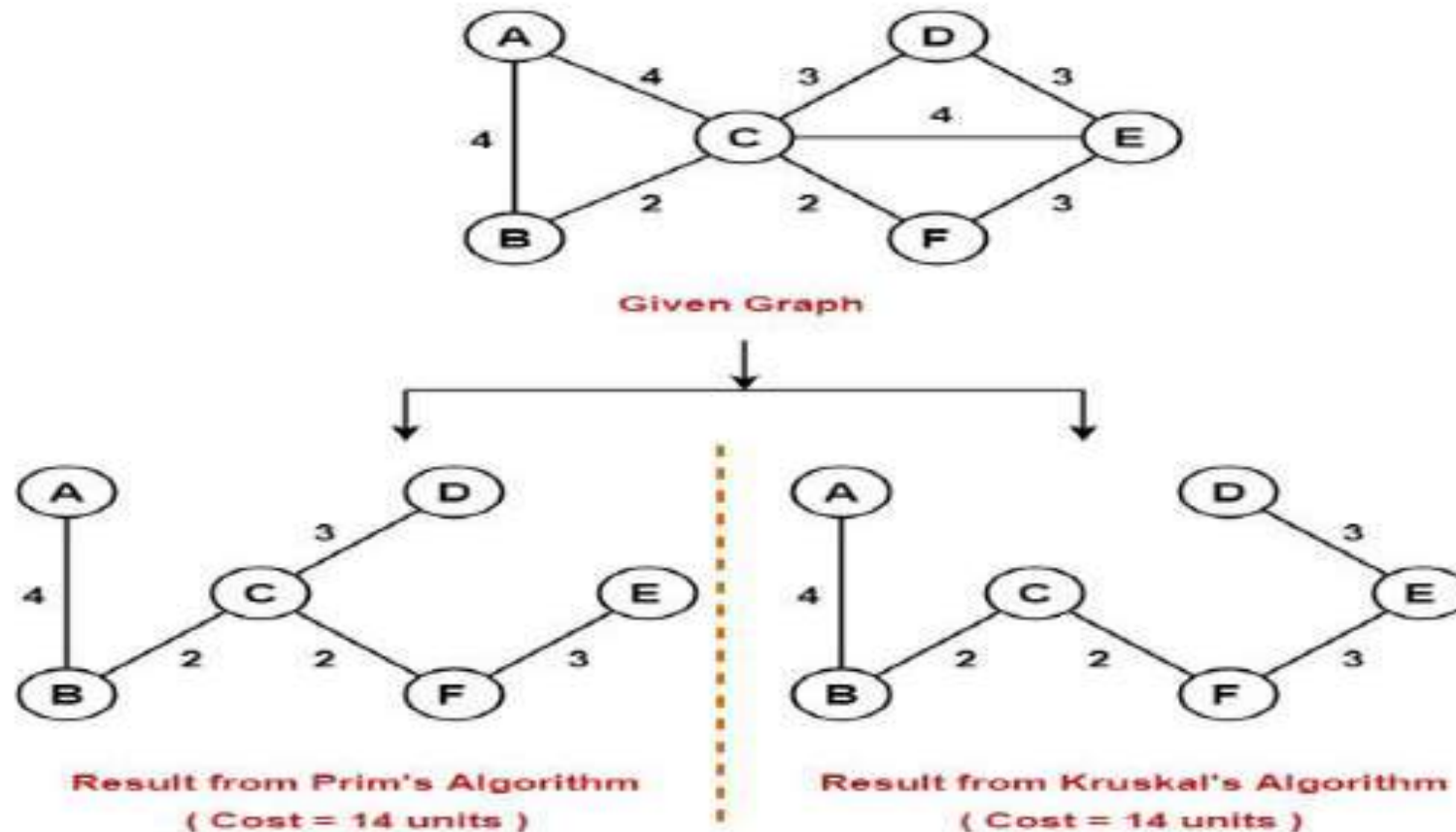
Case-02: When all the edge weights are not distinct-

Case-02: When all the edge weights are not distinct-

If all the edge weights are not distinct, then both the algorithms may not always produce the same i.e. unique MST but the cost of the MST produced would always be same in both the cases.

Example-

Consider the following example-



Differences



Kruskal's Algorithms	Prim's Algorithm
1. Select the shortest edge in a network 2. Select the next shortest edge which does not create a cycle 3. Repeat step 2 until all vertices have been connected 4. Kruskal's Begins with forest and merge into tree.	1. Select any vertex 2. Select the shortest edge connected to that vertex 3. Select the shortest edge connected to any vertex already connected 4. Repeat step 3 until all vertices have been connected 5. Prim's always stays as a tree.
<u>Complexity</u> $O(N \log N)$ comparison sort for edges.	<u>Complexity</u> $O(N \log N)$ search the least weight edge for every vertices.

SUMMARY

- A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..
- By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.
- Prim's and Kruskal's algorithm are used to construct the minimal spanning tree for a given graph topology.

SELF-ASSESSMENT QUESTIONS

Which data structure is commonly used to implement Prim's algorithm?

- (a) Array
- (b) Linked List
- (c) **Heap**
- (d) Stack

In Kruskal's algorithm, edges are added to the spanning tree in which order?

- (a) Decreasing order of weights.
- (b) Random order.
- (c) **Non-decreasing order of weights**
- (d) Alphabetical order.

TERMINAL QUESTIONS

1. Compare and contrast Prim's algorithm and Kruskal's algorithm for finding the MST of a weighted graph.
1. Explain the concept of cycle detection in the context of MST algorithms. Why is it important to avoid cycles when constructing a minimum spanning tree, and how do Prim's and Kruskal's algorithms ensure acyclic structures?

REFERENCES FOR FURTHER LEARNING OF THE SESSION

Reference Books :

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein., 3rd, 2009, The MIT Press.
- 2 Algorithm Design Manual, Steven S. Skiena., 2nd, 2008, Springer.
- 3 Data Structures and Algorithms in Python, Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser., 2nd, 2013, Wiley.
- 4 The Art of Computer Programming, Donald E. Knuth, 3rd, 1997, Addison-Wesley Professiona.

MOOCS :

1. <https://www.coursera.org/specializations/algorithms?=>
2. <https://www.coursera.org/learn/dynamic-programming-greedy-algorithms#modules>

THANK YOU